

UNIVERSITATEA BABEȘ-BOLYAI
CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI
INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ROMÂNĂ

LUCRARE DE LICENȚĂ

FileO, Orchestrator agentic de
fișiere

Conducător științific
Asist. dr. Briciu Anamaria

Absolvent
Vidami Alexandru-Constantin

2026

ABSTRACT

The rapid advancement of technology, both in software and hardware, has enabled a new generation of users to own personal computers on which they store an ever-growing collection of files. As a consequence, the need for reliable and accessible tools to manage, convert, and process those files has grown considerably, becoming increasingly demanded. In response to this trend, and leveraging the recent progress in artificial intelligence, this work presents *FileO*, a file orchestration toolbox designed to handle a wide variety of file types, including images, videos, and audio, through the use of an intelligent planning agent. The application also incorporates a virtual drive system built on the Windows NTFS filesystem and its Master File Table, enabling fast and structured access to all files stored on the machine. This paper examines the theoretical foundations of the problem, the architectural decisions made during design, the methodology used for testing, and the implementation details of the application, with the overarching goal of providing users with a more capable and intuitive solution for working with large collections of files.

Cuprins

1	Introducere	1
2	Memorarea Fișierelor în Sistemele de Windows	3
2.1	New Technology File System(NTFS)	3
2.2	Master File Table (MFT)	4
2.2.1	Structura Antetului Înregistrării MFT (Record Header)	5
2.2.2	Atributele Residente și Non-Residente	6
2.2.3	Atribute importante	6
2.3	Mecanisme de Identificare a Tipului de Date	7
2.4	Unelte Software de Lucru cu Fișiere	8
2.4.1	Analiza și Manipularea Documentelor	8
2.4.2	Procesarea Fluxurilor Multimedia (Audio și Video)	9
2.4.3	Analiza Obiectelor Vizuale (2D și 3D)	10
3	Inteligența Artificială Agentică	14
3.1	Cursul Progresului al Sistemelor Automatizate	14
3.2	Agentul Intelligent	16
3.2.1	Comparații cu Agenți Clasici	16
3.2.2	Fundamente tehnologice	17
3.2.3	Capabilitățile Agentului	18
3.3	Aplicații ale Inteligenței Agentice	19
3.3.1	Aplicații în Dezvoltarea Software	19
3.3.2	Aplicații în Procesele Business	19
3.3.3	Aplicații Industriale	20
3.4	Scalabilitate	20
3.5	Încrederea și Fiabilitatea	21
3.5.1	Robustețea Adversarială	21
3.5.2	Construirea unui Agent Onest	22
3.5.3	Alte riscuri potențiale	23

4	Abordări Anterioare în Industrie	24
4.1	Microsoft Copilot	24
4.2	Claude Cowork	25
4.3	WizTree	25
4.4	Site-uri de Lucru cu Fișiere	26
4.5	Abordarea Soluției	27
5	FileO, Aplicație de Orchestrare a Fișierelor cu Ajutorul unui Agent Inteligent	28
5.1	Specificarea Aplicației	28
5.1.1	Cerințe Funcționale	28
5.1.2	Cerințe Non-Funcționale	30
5.2	Proiectarea Aplicației	31
5.2.1	Arhitectura generală	31
5.2.2	Diagrame de clase	32
5.2.3	Diagrame de secvență	33
5.3	Implementare	35
5.3.1	Tehnologii utilizate	35
5.3.2	Pornirea aplicației și ciclul de viață	36
5.3.3	Executorul de instrumente și aprobarea umană	36
5.3.4	Procesarea fișierelor și modurile de output	37
5.3.5	Drive-ul virtual	39
5.3.6	Comunicarea dintre agent și aplicație	40
5.4	Testare	40
5.4.1	Performanța componentelor AI	40
5.4.2	Suite PyUnit pentru instrumente	41
6	Server pentru Agent și Autentificarea Clienților. Arhitectură și Implementare	43
6.1	Specificarea Aplicației	43
6.1.1	Cerințe Funcționale	43
6.1.2	Cerințe Non-Funcționale	46
6.2	Proiectarea sistemului	47
6.2.1	Arhitectura generală	47
6.2.2	Diagrame de clase	48
6.2.3	Diagrame de secvență	50
6.3	Implementare	51
6.3.1	Tehnologii utilizate	51
6.3.2	Modelul de date și persistența	52
6.3.3	Sistemul de autentificare	53

6.3.4	Gateway-ul de modele AI	53
6.3.5	Sistemul de agent cu planificare	54
6.3.6	Interfața web de administrare	57
6.4	Testare	57
7	Manualul Utilizatorului	59
7.1	Accesul la Aplicație	59
7.1.1	Autentificarea pe Server	59
7.1.2	Instalarea Aplicației	60
7.1.3	Acces la Interfața API	61
7.2	Secțiunile din Aplicație	62
7.2.1	Setări Inițiale	62
7.2.2	Drive Tab	62
7.2.3	Agent Tab	63
7.2.4	Tool Tab	64
8	Concluzii	66
	Bibliografie	68

Listă de figuri

2.1	Structura MFT [AVW09]	5
2.2	Schema Arhitecturală a Rețelei U ² -Net [QZH ⁺ 20]	12
2.3	Etapele procesului de transformare a imaginii în format vectorial prin algoritmul Potrace [Sel03]	13
3.1	Evoluția paradigmelor de AI [AAD25]	14
3.2	Ilustrare a componentelor Inteligenței Agentice [AKD25]	17
5.1	Arhitectura proceselor aplicației FileO	32
5.2	Structura modulelor de instrumente (backend)	33
5.3	Ierarhia componentelor React (frontend)	33
5.4	Secvența de execuție directă a unui instrument	34
5.5	Secvența de execuție a unui instrument prin agent (human-in-the-loop)	35
6.1	Diagrama Use Case UC-1: Autentificare și gestionare cont	45
6.2	Diagrama Use Case UC-2: Gateway AI	46
6.3	Diagrama Use Case UC-3: Sistemul de agent inteligent	46
6.4	Diagrama de componente a serverului	47
6.5	Diagrama de deployment: infrastructura curentă	48
6.6	Diagrama de clase: modelul de date ORM	48
6.7	Diagrama de clase: ierarhia modelelor AI	49
6.8	Diagramă de clase: Sistemul de agent (PlanningAgent)	49
6.9	Diagrama de secvență: autentificare JWT	50
6.10	Diagrama de secvență: agentul cu planificare Plan-and-Execute	51
6.11	Diagrama de secvență: Trecerea pipeline a fazelor agentului	56
7.1	Pagina de autentificare a serverului	59
7.2	Descărcarea aplicației FileO din interfața web	60
7.3	Fereastra principală a aplicației FileO la prima pornire	60
7.4	Generarea cheii API din interfața web a serverului	61
7.5	Configurarea cheii API în aplicația FileO	61
7.6	Secțiunea My Drives, vizualizare drive virtual	62

7.7	Secțiunea Agent, conversație activă cu agentul de planificare	63
7.8	Dialogul de aprobare a execuției unui instrument	64
7.9	Secțiunea Tools, grila de instrumente organizată pe categorii	65
7.10	Formularul de execuție al unui instrument	65
7.11	Rezultatele după execuția unui instrument	65

Listă de tabele

2.1	Structura înregistrării MFT bazată pe specificațiile NTFS 3.1.	5
3.1	Comparația între agenții clasici, agenții RL și Agentic AI [AKD25] . . .	17
5.1	Categoriile de instrumente ale aplicației FileO	29
5.2	Câmpurile instrumentului <i>Image Converter</i>	29

Capitolul 1

Introducere

În perioada actuală, spre deosebire de acum un deceniu sau două, accesul la un calculator personal a devenit aproape universal. Odată cu această democratizare a tehnologiei, volumul de fișiere pe care un utilizator obișnuit le gestionează a crescut semnificativ: imagini, înregistrări video, fișiere audio, documente de lucru sau arhive personale reprezintă astăzi o parte integrantă a vieții digitale cotidiene. Gestionarea eficientă a acestor resurse, convertirea, organizarea, analiza sau transformarea lor, constituie o nevoie reală și frecventă, căreia instrumentele tradiționale îi răspund doar parțial, impunând adesea un efort manual repetitiv și cunoștințe tehnice specializate.

Totodată, progresul tehnologic din ultimii ani a adus în prim-plan inteligența artificială agentică, o nouă paradigmă în care sistemele AI nu mai răspund pasiv la întrebări, ci planifică și execută secvențe de acțiuni pentru atingerea unui obiectiv. Această evoluție creează posibilitatea integrării inteligenței artificiale direct în fluxurile de lucru cu fișierele, transformând modul în care utilizatorii interacționează cu datele lor.

Pornind de la aceste premise, lucrarea de față propune proiectarea și implementarea aplicației *FileO*, un orchestrator agentic de fișiere care pune la dispoziția utilizatorului un set de instrumente specializate de procesare, coordonate de un agent inteligent capabil să înțeleagă cerințe exprimate în limbaj natural, să elaboreze un plan de execuție și să îl ducă la îndeplinire în mod automat, cu posibilitatea intervenției umane în pașii critici.

Contribuția originală a lucrării rezidă se referă la conceperea și implementarea unei arhitecturi care se distinge de soluțiile existente din industrie prin mai multe aspecte. În timp ce platformele comerciale de tip cloud (Adobe, Google Photos, online converters) oferă instrumente izolate, fără capacitate de raționament sau înlănțuire automată a acțiunilor, iar asistenții AI generalişti (ChatGPT, Gemini) nu pot opera direct asupra sistemului de fișiere local, FileO propune o abordare hibridă: un set de instrumente de procesare modulare, descrise fiecare printr-o schemă JSON formală, coordonate de un agent de planificare propriu, construit fără dependența de framework-uri de orchestrare externe. Agentul urmează un pipeline personalizat Plan–Execute–Synthesize, poate

recunoaște mesajele de follow-up și evită reapelarea inutilă a instrumentelor, beneficiind totodată de un mecanism de aprobare umană la pașii critici. În plus, integrarea drive-urilor virtuale bazate pe tehnologia NTFS și Master File Table a sistemului Windows oferă utilizatorului o vizualizare organizată a fișierelor de pe disc, independent de locația fizică a acestora, o funcționalitate absentă din soluțiile curente de pe piață.

Lucrarea este structurată în opt capitole, acesta fiind primul capitol cu rol introductiv. Capitolul 2 aprofundează modul în care sistemul de operare Windows gestionează fișierele la nivel fizic, cu accent pe sistemul de fișiere NTFS, structura Master File Table și mecanismele de identificare a tipului de date, oferind contextul tehnic necesar pentru înțelegerea instrumentelor de procesare implementate. Capitolul 3 acoperă domeniul inteligenței artificiale agentice: evoluția paradigmelor de AI, arhitectura și capabilitățile unui agent inteligent modern, aplicații reprezentative din diverse domenii, precum și aspecte legate de scalabilitate, încredere și potențiale riscuri. Capitolul 4 prezintă abordările anterioare din industrie.

Capitolul 5 și Capitolul 6 constituie nucleul tehnic al lucrării: primul detaliază aplicația client FileO, specificații, proiectare, tehnologii și implementarea componentelor principale (executor de instrumente, drive virtual, comunicarea cu agentul), iar al doilea descrie serverul backend responsabil de autentificarea clienților, gateway-ul de modele AI și agentul de planificare cu pipeline-ul Plan–Execute–Synthesize. Capitolul 7 reprezintă manualul utilizatorului, ghidând instalarea aplicației și utilizarea funcționalităților principale.

Lucrarea se încheie cu Capitolul 8, care sintetizează contribuțiile originale aduse, prezintă concluziile desprinse în urma implementării și testării sistemului și propune direcții concrete de dezvoltare ulterioară, atât la nivelul instrumentelor de procesare, cât și al capabilităților agentului inteligent.

Toate sursele utilizate în elaborarea prezentei lucrări sunt citate și referențiate în secțiunea de bibliografie, în conformitate cu normele academice în vigoare.

În redactarea lucrării, au fost utilizate instrumentele de inteligență artificială, Gemini și Claude, care au grăbit viteza de lucru în activitatea de experimentare cu tehnologiile folosite. Conținutul generat a fost revizuit critic și editat de autor, care își asumă întreaga responsabilitate pentru originalitatea și acuratețea lucrării.

Capitolul 2

Memorarea Fișierelor în Sistemele de Windows

Necesitatea stocării datelor pe termen lung a condus la dezvoltarea sistemelor de fișiere, mecanisme complexe care permit utilizatorilor să gestioneze o ierarhie structurată de fișiere și directoare. Aceste sisteme sunt compuse din metadate structurale și date de utilizator, organizate riguros pentru a optimiza procesul de regăsire a informațiilor. Prezentul capitol analizează modul în care sistemele de operare Windows orchestrează aceste elemente prin intermediul mecanismului propriu, intitulat New Technology File System (NTFS) [Car05].

2.1 New Technology File System(NTFS)

Sistemul de fișiere New Technology File System (NTFS) a fost proiectat de către Microsoft și reprezintă sistemul de fișiere implicit pentru versiunile de sistem de operare Microsoft Windows NT, Windows 2000, Windows XP și Windows Server.

NTFS a fost conceput pentru a oferi fiabilitate, securitate și suport pentru dispozitive de stocare de mari dimensiuni. Scalabilitatea este asigurată prin utilizarea unor structuri de date generice care încapsulează structuri de date cu conținut specific. Acesta este un design scalabil, deoarece structura internă a datelor se poate modifica în timp pe măsură ce apar noi cerințe asupra sistemului de fișiere, în timp ce învelișul general poate rămâne constant. Un exemplu de înveliș generic este faptul că fiecare octet de date dintr-un sistem de fișiere NTFS este alocat unui fișier.

NTFS este un sistem de fișiere complex și, din nefericire, nu există o specificație publicată de către Microsoft care să descrie dispunerea datelor pe disc. Deși au fost publicate descrieri la nivel înalt ale componentelor sistemului de fișiere, detaliile de nivel scăzut sunt rare.

NTFS este standard pe multe sisteme Windows și devine tot mai utilizat în majori-

tatea distribuțiilor Unix gratuite. Combinația dintre absența unei specificații oficiale și existența unei singure aplicații dominante care creează sistemul de fișiere face dificilă diferențierea între proprietățile specifice aplicației și proprietățile generale ale sistemului de fișiere. De exemplu, există alte metode care ar putea fi utilizate pentru a inițializa un sistem de fișiere pe care Microsoft nu le folosește și nu este clar dacă acestea ar trebui considerate sisteme de fișiere "NTFS valide".

Unul dintre cele mai importante concepte în înțelegerea arhitecturii NTFS este faptul că datele relevante sunt alocate fișierelor. Aceasta include datele administrative de bază ale sistemului de fișiere, care sunt de obicei ascunse de alte sisteme de fișiere. În fapt, fișierele care conțin datele administrative pot fi localizate oriunde în cadrul volumului, la fel ca un fișier obișnuit. Prin urmare, un sistem de fișiere NTFS nu are o dispunere specifică, spre deosebire de alte sisteme de fișiere. Întregul sistem de fișiere este considerat o zonă de date, iar orice sector poate fi alocat unui fișier. Singura dispunere constantă este reprezentată de faptul că primele sectoare ale volumului conțin sectorul de boot și codul de boot.

2.2 Master File Table (MFT)

Elementul central al arhitecturii NTFS este Master File Table (MFT), o structură fundamentală ce conține metadatele aferente fiecărui fișier și director de pe volum, alocând în mod uzual două sectoare pentru fiecare intrare. Prin urmare, o cunoaștere detaliată a topologiei MFT este esențială nu doar pentru înțelegerea sistemului, ci și pentru eficientizarea proceselor de recuperare a datelor de pe suporturile de stocare.

Fiecare intrare din tabelul MFT are o dimensiune fixă de 1 KB (parametru ce poate fi identificat la nivelul sectorului de boot, la offset-ul de 64 octeți). Figura 2.1 ilustrează structura generală a MFT și planul logic al sistemului de fișiere NTFS. Obiectivul principal al NTFS este optimizarea operațiunilor de citire și scriere a atributelor fișierelor, MFT permițând analistului forensic să examineze minuțios structura și mecanismele operaționale ale volumului. Din acest considerent, este imperativă înțelegerea modului în care attributele sunt stocate în cadrul unei intrări MFT.

O caracteristică definitorie a acestor intrări este versatilitatea atributelor conținute, care pot varia ca format și dimensiune. După cum este evidențiat în figura 2.1, fiecare atribut include un antet (header), localizat în primii 42 de octeți ai înregistrării, care servește la identificarea dimensiunii, numelui și a valorilor de tip flag. În funcție de dimensiunea datelor, conținutul atributului poate fi stocat direct în MFT, imediat după antet, dacă acesta nu depășește pragul de aproximativ 700 de octeți (caz în care atributul este definit ca fiind rezident). În situația în care dimensiunea datelor excede capacitatea disponibilă a intrării de 1 KB, conținutul este stocat în clustere externe, structură cunoscută sub numele de "cluster run" (atribut nerezident). Această distincție

este determinată de limitarea fizică a intrării MFT, care nu poate găzdui atribute a căror dimensiune totală depășește spațiul util rămas în interiorul înregistrării.

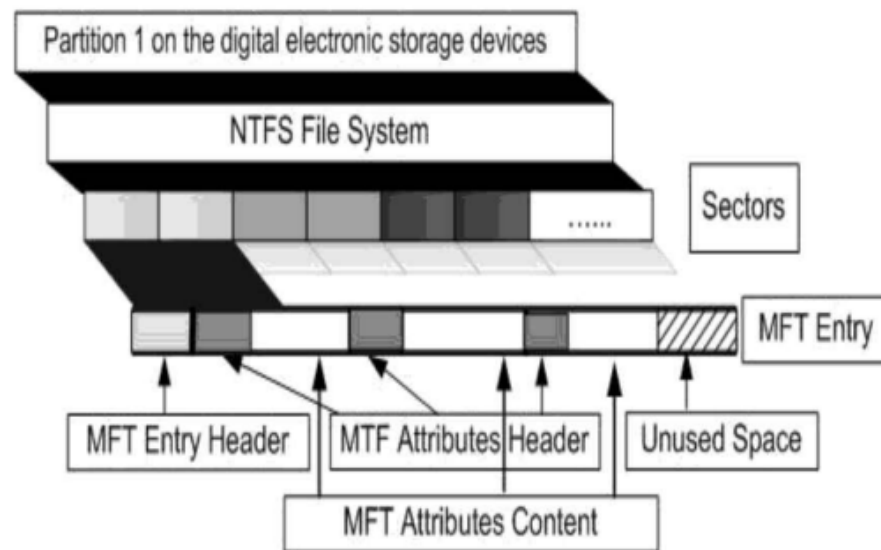


Figura 2.1: Structura MFT [AVW09]

2.2.1 Structura Antetului Înregistrării MFT (Record Header)

Fiecare înregistrare din Master File Table (\$MFT) începe cu un antet fix de 56 de octeți (0x38), care conține metadate critice despre structura și starea înregistrării respective. Dimensiunea fizică a unei înregistrări MFT este stabilită la crearea volumului (formatare), cele mai comune valori fiind de 1024 sau 4096 octeți. Structura detaliată a antetului este prezentată în tabelul de mai jos:

0x00	4	Semnătura înregistrării (de regulă "FILE")
0x10	2	Numărul secvenței înregistrării MFT
0x12	2	Numărul de legături fizice (HardLink Count)
0x14	2	Offset către primul atribut (relativ la începutul recordului)
0x16	2	Flag-uri de stare a alocării (ex. 0x01 = În uz, 0x02 = Director)
0x18	4	Dimensiunea logică a înregistrării MFT
0x1C	4	Dimensiunea fizică a înregistrării MFT
0x20	4	Numărul înregistrării MFT (partea inferioară)

Tabela 2.1: Structura înregistrării MFT bazată pe specificațiile NTFS 3.1.

Un element de siguranță important inclus în antet este mecanismul de *fixup* (Update Sequence Array), utilizat pentru a detecta coruperea sectoarelor. Numărul de perechi de octeți pentru fixup depinde de dimensiunea înregistrării: pentru 1024 octeți sunt necesare 3 perechi, iar pentru 4096 octeți sunt necesare 9 perechi.

2.2.2 Atributele Residente și Non-Residente

În sistemul NTFS, toate datele (inclusiv conținutul fișierelor) sunt stocate sub formă de atribute. Acestea sunt clasificate în două categorii, în funcție de locația stocării lor:

1. **Atribute Residente:** Atunci când conținutul atributului este suficient de mic pentru a încăpea în spațiul rămas din înregistrarea MFT (după antet), acesta este stocat direct acolo. Statusul rezidenței este indicat de un flag de 1 octet aflat la offset-ul 0x08 în antetul atributului (valoarea 0x00 indică un atribut rezident).
2. **Atribute Non-Residente:** Dacă datele sunt prea mari, acestea sunt stocate în clustere externe din afara tabelului MFT. În înregistrarea MFT rămâne doar un antet care conține o „hartă” a locațiilor externe numită *Datarun*. Valoarea 0x01 în flag-ul de status indică faptul că atributul este non-rezident.

Toate atributele încep cu un antet comun care definește tipul atributului, lungimea acestuia și identificatorul unic.

2.2.3 Atribute importante

Înregistrările MFT conțin diverse tipuri de atribute, fiecare având un rol specific în gestionarea fișierului:

- **\$STANDARD_INFORMATION (0x10):** Este întotdeauna rezident și conține informații de bază precum marcajele temporale (creare, modificare, acces), atributele de fișier (ReadOnly, Hidden, System) și ID-urile de securitate.
- **\$FILE_NAME (0x30):** Atribut rezident care stochează numele fișierului, dimensiunea alocată pe disc și referința către directorul părinte. Un fișier poate avea multiple atribute de acest tip (de exemplu, pentru hard links sau nume scurte de tip DOS).
- **\$DATA (0x80):** Reprezintă conținutul propriu-zis al fișierului. Acesta poate fi rezident pentru fișiere foarte mici sau non-rezident pentru majoritatea datelor de utilizator.
- **\$ATTRIBUTE_LIST (0x20):** Utilizat atunci când un fișier are atât de multe atribute încât acestea nu mai încap într-o singură înregistrare MFT; acest atribut servește ca index către înregistrările MFT adiționale (child records).
- **\$INDEX_ROOT (0x90) și \$INDEX_ALLOCATION (0xA0):** Aceste atribute sunt fundamentale pentru structura directoarelor, implementând arborii B-tree utilizați pentru căutarea rapidă a fișierelor.

Fiecare atribut primește un ID unic determinat de ordinea în care este scris în înregistrare. Sfârșitul unei înregistrări MFT valide este întotdeauna marcat de semnătura specială 0xFFFFFFFF.

2.3 Mecanisme de Identificare a Tipului de Date

Un fișier poate fi definit drept o entitate informațională reprezentată printr-o secvență de octeți, structurată conform unei convenții sau unui format specific, propus de un autor sau de un consorțiu tehnic. În mediile de operare de tip Windows, fișierele sunt identificate printr-un nume extern compus, de regulă, din două segmente: denumirea propriu-zisă, care facilitează accesul utilizatorului, și extensia, care indică tipul de date conținut. Prin intermediul extensiei, sistemul de operare determină software-ul adecvat pentru procesarea fișierului, utilizând o tabelă de asocieri care corelează extensiile cu aplicațiile instalate.

Utilizarea extensiilor reprezintă o metodă facilă de identificare a tipurilor comune de date. Anumite extensii sunt atât de larg răspândite încât probabilitatea ca acestea să corespundă altor formate este minimă; exemple relevante includ .exe pentru fișiere executabile, .txt pentru documente text simplu, sau .jpg, .png și .gif pentru formate de imagine. Cu toate acestea, dependența exclusivă de extensii în procesul de categorisire prezintă limitări semnificative:

1. Sinonimia extensiilor: Mai multe extensii pot referi același format de fișier (ex: .jpg și .jpeg).
2. Divergența sistemelor de operare: Anumite platforme, precum macOS, nu depind de prezența extensiei pentru lansarea aplicațiilor, utilizând registre interne pentru a stabili asocierea dintre fișier și software.
3. Conflicte de nomenclatură: Aplicații diferite pot utiliza aceeași extensie pentru scopuri structurale divergente.
4. Manipularea intenționată: Un utilizator poate modifica voluntar extensia unui fișier pentru a evita categorisirea automată sau pentru a oculta natura datelor.

Pentru a uniformiza convențiile de denumire și a gestiona absența extensiilor în schimbul de informații, a fost introdus standardul MIME (Multipurpose Internet Mail Extensions) [FB96]. Acesta include câmpul Content-Type, care oferă o modalitate standardizată de declarare a tipurilor media, gestionată de IANA (Internet Assigned Numbers Authority) ¹. Extensia și tipul de conținut MIME reprezintă metadata

¹IANA Media Types

exogene (externe), a căror modificare nu corupe integritatea structurală a fișierului, acesta rămânând lizibil pentru aplicația corectă.

O metodă superioară și mai fiabilă de detecție constă în identificarea semnăturilor de fișier, cunoscute sub denumirea de "magic numbers". Acestea sunt secvențe fixe de octeți integrate în reprezentarea binară a fișierului, localizate de obicei în antet (header)². Aceste semnături pot reprezenta acronime ASCII ale formatului (ex: 0x50 0x4E 0x47 pentru PNG sau 0x25 0x50 0x44 0x46 0x2D pentru PDF) ori inițialele creatorului formatului (ex: 0x50 0x4B pentru formatul ZIP, referindu-se la Phil Katz).

Identificarea bazată pe semnături binare este considerabil mai riguroasă decât verificarea extensiei, fiind o componentă intrinsecă a structurii fișierului. Bibliotecile de procesare (parsing) validează aceste semnături la începutul procesului de decodare; alterarea acestora conduce la imposibilitatea citirii fișierului. Din acest motiv, actorii interesați de ocultarea informațiilor preferă modificarea metadatelor exogene pentru un "camuflaj superficial" sau utilizarea tehnicilor criptografice și steganografice pentru o ocultare profundă. În scenariile de recuperare a datelor de pe partiții corupte, unde metadatele directoarelor sunt pierdute, instrumentele specializate utilizează aceste semnături binare pentru a reconstitui tipul fișierelor și a le atribui extensiile corecte. Deși nu există o garanție absolută împotriva coliziunilor între o semnătură validă și o secvență arbitrară de octeți, utilizarea "magic numbers" reprezintă cel mai bun compromis tehnologic pentru detecția fiabilă a tipurilor de date [DGGR23].

2.4 Unelte Software de Lucru cu Fișiere

Având în vedere diversitatea formatelor de date, utilizatorii necesită instrumente și servicii specializate pentru îndeplinirea sarcinilor specifice fiecărui tip de fișier. Printre aceste operațiuni se numără conversia documentelor între diferite extensii din aceeași categorie, precum și extragerea informațiilor din fluxuri audio sau video sub formă de text. În secțiunile următoare, vor fi prezentate diverse soluții software și metodologiile aferente, unele dintre acestea utilizând algoritmi de Inteligență Artificială pentru procesarea datelor, care au fost integrate sau testate în cadrul ecosistemului aplicației *FileO*.

2.4.1 Analiza și Manipularea Documentelor

Gestionarea programatică a documentelor în formate eterogene reprezintă o problemă fundamentală în ingineria documentelor, abordată sistematic încă de la primele ediții ale simpozionului ACM DocEng. Lucrarea prezentată la ACM Symposium on Document Engineering [LB02] evidențiază necesitatea unor mecanisme formale de transformare a

²Tabelă cu semnături de fișier

documentelor structurate, subliniind că interoperabilitatea între formate nu poate fi redusă la o simplă substituție sintactică, ci implică păstrarea semanticii și a structurii logice a conținutului. Această perspectivă a stat la baza proiectării componentelor de procesare a documentelor din cadrul sistemului de față, unde conversia între formate trebuie să respecte invarianți structurali indiferent de direcția transformării.

2.4.2 Procesarea Fluxurilor Multimedia (Audio și Video)

Fișierele audio digitalizează semnalul sonor continuu printr-un proces de eșantionare și cuantizare, rezultând un șir de valori PCM (*Pulse-Code Modulation*) stocate brut sau supuse ulterior unui algoritm de compresie. Calitatea redării, dimensiunea fișierului și gradul de interoperabilitate depind în mod direct de formatul ales, fapt pentru care literatura de specialitate distinge trei categorii principale: necomprimat, comprimat fără pierderi și comprimat cu pierderi.

Formatele necomprimate și cele *lossless* păstrează integral informația sonoră originală. **WAV** (*Waveform Audio File Format*), standardizat de Microsoft și IBM la începutul anilor 1990, stochează datele PCM brute și rămâne referința de facto în producția audio profesională, cu toate că ocupă aproximativ 10,6 MB per minut la parametrii CD (44,1 kHz, 16 biți, stereo). **FLAC** (*Free Lossless Audio Codec*), lansat de Fundația Xiph.Org în 2001 și formalizat ca standard IETF prin RFC 9639 [dCLCT⁺24], realizează o reducere a dimensiunii la 50–70% din fișierul WAV echivalent prin codare predictivă și entropică, fără a elimina nicio informație sonoră. Reconstrucția la decodare este identică bit cu bit față de sursa originală.

Atunci când restricțiile de lățime de bandă sau de stocare primează, codecurile *lossy* oferă dimensiuni de fișier cu un ordin de mărime mai mici, prin eliminarea componentelor spectrale considerate imperceptibile conform modelelor psihoacustice. **MP3** (*MPEG-1 Audio Layer III*) [Bra99], al cărui algoritm se bazează pe lucrările lui Karlheinz Brandenburg și a fost standardizat prin ISO/IEC 11172-3 în 1993, poate reduce un fișier audio cu până la 90–95% față de echivalentul necomprimat, reținând exclusiv frecvențele relevante pentru percepția umană. **AAC** (*Advanced Audio Coding*) [BG03] a fost conceput ca succesor al MP3, oferind o calitate superioară la rate de biți comparabile printr-un model psihoacustic mai rafinat și o bancă de filtre îmbunătățită. Mai recent, **Opus** [VVT12], standardizat de IETF prin RFC 6716 în 2012, unifică într-un singur codec capacitățile necesare atât comunicațiilor în timp real, cât și arhivării la fidelitate ridicată, fără a impune nicio restricție de patent sau licențiere.

Fișierele video încapsulează secvențe de cadre comprimate împreună cu unul sau mai multe fluxuri audio, sincronizate printr-un container (precum *MP4*, *MKV* sau *WebM*). Spre deosebire de imaginile statice, compresia video exploatează nu doar redundanța spațială din interiorul fiecărui cadru, ci și redundanța temporală dintre cadre consecutive,

cea ce face din alegerea codecului o decizie cu implicații profunde asupra calității, latenței și costului de stocare.

Codecurile video fără pierderi (*lossless*) garantează că fiecare cadru poate fi reconstruit identic din datele comprimate. Această proprietate le recomandă în fluxurile de postproducție și arhivare, unde orice degradare a imaginii este inacceptabilă. **Apple ProRes** și **Avid DNxHD** reprezintă soluțiile consacrate în industria cinematografică și televiziunii profesionale: ambele aplică o compresie intraframe redusă, menținând o fidelitate ridicată față de semnalul brut captat de cameră, cu prețul unor dimensiuni de fișier considerabile.

Utilizarea în distribuție și streaming a impus dezvoltarea unor codecuri *lossy* capabile să reducă substanțial debitul binar fără a compromite calitatea percepută de spectator. Standardul **H.264/AVC** [WSBL03], elaborat în comun de ITU-T și ISO/IEC și finalizat în 2003, a revoluționat domeniul prin combinarea predicției de mișcare adaptive cu transformate pe blocuri mici și un filtru de deblochizare în buclă, obținând o reducere de aproximativ 50% a debitului față de standardele anterioare. Succesorul său, **H.265/HEVC** [SOHW12], atinge un raport calitate-compresie cu circa 50% superior față de H.264 la aceeași calitate perceptuală, fapt care l-a impus în scenariile de distribuție 4K și HDR. Alternativa deschisă la aceste standarde licențiate este reprezentată de **VP9** [MBG⁺13], dezvoltat de Google ca format fără redevențe și adoptat la scară largă de YouTube, și de **AV1** [CMH⁺18], creat de Alliance for Open Media, care aduce o eficiență comparabilă cu H.265 în absența oricărei taxe de licențiere.

2.4.3 Analiza Obiectelor Vizuale (2D și 3D)

Formatele fișierelor de imagine pot fi clasificate, în sens larg, în funcție de regimul de comprimare (cu pierderi sau fără pierderi) și de natura licențierii acestora (formate deschise sau proprietare).

Formatele de imagine de tip *lossless* sunt concepute astfel încât datele originale ale imaginii să fie stocate intact sau comprimate prin algoritmi care permit reconstrucția identică a informației la decompresie. Prin urmare, aceste fișiere oferă o fidelitate superioară față de imaginea sursă, însă implică un cost ridicat în ceea ce privește volumul de stocare. Exemple reprezentative includ formatul **TIFF** (stocat necomprimat în forma sa canonică) și **PNG**, care utilizează un mecanism de compresie fără pierderi.

Spre deosebire de primele, formatele *lossy* utilizează algoritmi de compresie care reduc dimensiunea fișierului prin eliminarea selectivă a anumitor date de imagine. Acest proces permite obținerea unor fișiere de dimensiuni reduse în comparație cu formatele *lossless*, cel mai proeminent exemplu fiind standardul **JPEG**. De asemenea, printre evoluțiile recente se numără formatul **JPEG2000**, care oferă flexibilitatea de a opera atât în regim *lossy*, cât și *lossless*, în funcție de necesitățile aplicației.

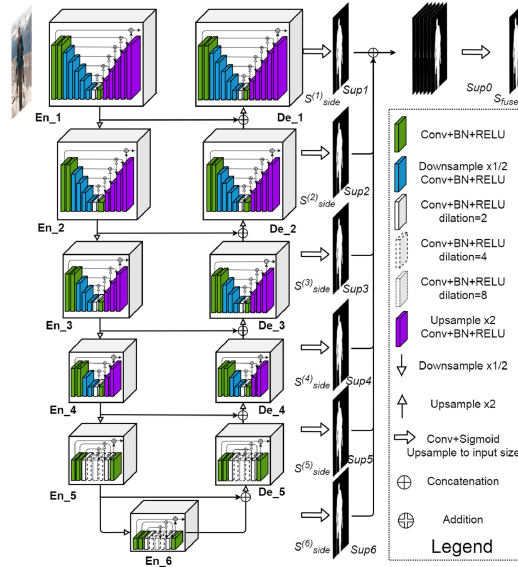
Eliminarea Fundalului.

Eliminare fundalului este realizat de librăria Python numită "rembg" [Gat23], bazată pe o arhitectură **U²-Net**, care se distinge printr-o structură ierarhică de tip "nested U-structure". Așa cum se observă în 2.2 rețeaua nu este formată din simple straturi de convoluție, ci din module complexe numite **RSU (Residual U-blocks)** [SY⁺24]. Spre deosebire de arhitecturile clasice care pierd detalii fine în procesul de eșantionare (downsampling), blocurile RSU (de la En_1 la En_6 și de la De_1 la De_5) permit extragerea caracteristicilor multi-scală fără a compromite rezoluția spațială. Această capacitate este critică pentru eliminarea fundalului, unde detaliile precum firele de păr sau marginile obiectelor complexe trebuie conservate riguros.

Procesul de segmentare începe în etapa de Encoder (coloana stângă din 2.2), unde imaginea este comprimată progresiv pentru a capta contextul semantic al obiectului proeminent. Ulterior, etapa de Decoder (coloana dreaptă) reface rezoluția imaginii prin operațiuni de upsampling și concatenare, marcate prin simbolul $\oplus$. Această simetrie asigură faptul că informația globală, responsabilă pentru conturul general, este combinată cu informația locală de înaltă rezoluție. Acest flux de date permite algoritmului să distingă cu precizie între subiectul principal și elementele de fundal complexe

Un aspect fundamental ilustrat în 2.2 este mecanismul de Deep Supervision. Rețeaua generează hărți de probabilitate (Saliency Map [Nie07]) intermediare la fiecare nivel al decodului, notate cu $Sup1$ până la $Sup6$. Fiecare dintre aceste hărți trece printr-o funcție de activare Sigmoid pentru a produce o mască de segmentare binară. Rezultatul final, reprezentat de hărțile $Sup0$ și S_{fuse} , constituie fuziunea tuturor acestor niveluri. În contextul instrumentului de procesare, această mască finală (alpha matte) determină exact care pixeli sunt păstrați ca făcând parte din subiect și care sunt transpuși în transparență.

În blocurile de profunzime (precum En_5 și En_6), figura indică utilizarea convoluțiilor dilatate (dilation=2, 4, 8). Această tehnică tehnologică permite mărirea câmpului receptiv al rețelei fără a crește numărul de parametri sau a pierde rezoluția prin pooling. Pentru utilizatorul final, acest lucru se traduce printr-o acuratețe ridicată în detectarea obiectelor mari din imagine, asigurându-se că nicio parte a subiectului nu este "ștearsă" accidental odată cu fundalul.

Figura 2.2: Schema Arhitecturală a Rețelei U²-Net [QZH+20]

Imagine transformată în SVG vector.

Algoritmul utilizat pentru transformarea imaginilor raster în formate vectoriale (SVG) se bazează pe metodologia **Potrace** [Sel03], care procesează hărțile de biți prin descompunerea acestora în trasee rectilinii și curbe. Procesul începe cu faza de descompunere în componente, unde imaginea este segmentată în regiuni negre și albe, formând contururi poligonale brute. Ulterior, algoritmul aplică o etapă de aproximare optimală, în care poligoanele sunt simplificate și transformate în curbe Bézier [BK19] netede. Acest lucru este realizat prin minimizarea unei funcții de cost care echilibrează precizia fidelității față de imaginea originală cu numărul minim de puncte de control necesare pentru a menține curba fluidă.

Spre deosebire de alte metode de vectorizare, abordarea descrisă de Selinger se remarcă prin independența de rezoluție, transformând pixelii discreți în entități geometrice continue. Această tehnologie permite obținerea unor fișiere SVG care pot fi scalate infinit fără pierderea calității sau apariția fenomenului de pixelare. Algoritmul implementează, de asemenea, tehnici de filtrare a zgomotului (speckle filtering), eliminând automat artefactele mici de compresie din imaginea sursă, rezultând astfel un contur vectorial curat, ideal pentru design grafic și ilustrații digitale.

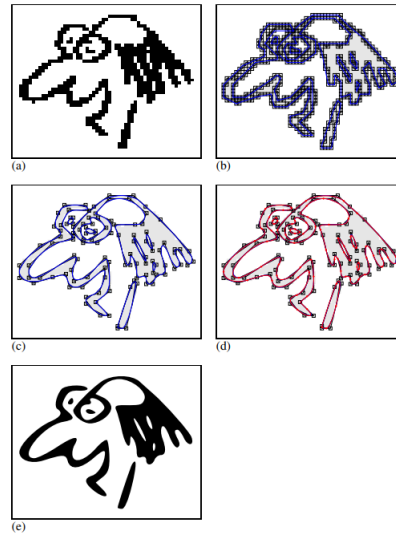


Figura 2.3: Etapele procesului de transformare a imaginii în format vectorial prin algoritmul Potrace [Sel03]

Obiectele tridimensionale [MB08]

Sunt clasificate în mod fundamental în trei categorii principale: geometrie, aspect și informații de scenă. Geometria sau forma unui model este stocată, în mod uzual, sub forma unui set de puncte 3D denumite vertice, suprafața fiind definită printr-o serie de poligoane sau fețe construite prin indexarea acestor puncte.

Proprietățile de aspect definesc modul în care suprafețele obiectelor 3D sunt redată vizual, cea mai utilizată metodă fiind aplicarea unei texturi prin maparea fiecărui vertex tridimensional la un punct corespondent dintr-o imagine bidimensională. Pe lângă texturare, aspectul poate fi definit prin modelarea proprietăților fizice ale suprafeței, incluzând componente difuze pentru culoare și reflexie, componente speculare pentru intensitatea reflexiilor în oglindă și componente transmise pentru materiale transparente sau semitransparente. Distorsiunea luminii prin suprafețele transparente este reprezentată printr-un indice de refracție, în timp ce emisia de lumină proprie a unui material este definită printr-o componentă emisivă.

Conceptul de scenă se referă la dispunerea spațială a modelului în raport cu camera, sursele de lumină și alte obiecte 3D. Definirea completă a camerei necesită parametri de magnificare, poziția 3D, un vector de direcție și un vector care indică orientarea verticală. Sursele de lumină sunt reprezentate, de regulă, prin vectori 3D care indică poziția în spațiu și intensitatea culorii. De asemenea, organizarea scenei permite desemnarea unor grupuri separate de obiecte și stocarea transformărilor aferente fiecărei părți prin matrice de transformare, facilitând reconstrucția corectă a ansamblului tridimensional.

Capitolul 3

Inteligența Artificială Agentică

În ultima perioadă domeniul Inteligenței Artificiale a fost dominat de progresul modelelor generative de tip Transformer [VSP⁺17] care au demonstrat capacități remarcabile de utilizarea limbajului natural și a puterii de sintetizare a informațiilor.

Inteligența Agentică reprezintă următorul pas în paradigma fluxului de muncă automatizat, evoluând de la sistemele autonome bazate pe reguli, la agenți capabili de raționament, planificare și executarea cerințelor cu supervizare umană minimală.

3.1 Cursul Progresului al Sistemelor Automatizate

Deși în ultimul deceniu a existat un progres vizibil în capacitățile de automatizare a unui număr mare de sarcini din diverse domenii, eforturile în acest sens datează încă din anii 1950. O privire de ansamblu asupra evoluției paradigmelor din domeniul inteligenței artificiale este prezentată în Figură 3.1.

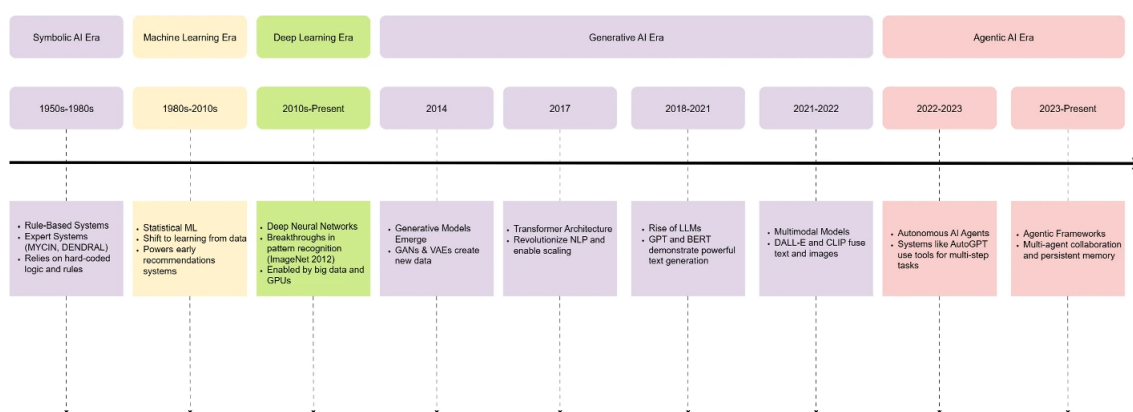


Figura 3.1: Evoluția paradigmelor de AI [AADC25]

Prima eră, numită era simbolică a inteligenței artificiale (1950-1980), a fost caracterizată în principal de dezvoltarea sistemelor bazate pe reguli, a căror elaborare presupunea expertiză în domeniul de aplicare prin sisteme bazate pe reguli a căror

creare presupunea expertiză în domeniu, cunoștințe umane, ce operau pe algoritm "hard-coded". Printre sistemele de acest tip acestea s-au numărat MYCIN [Sho12] și DENDRAL [BF81], fiind primele modele folosite în chimie organică și luarea de decizii folosindu-se de cunoștințele dobândite de oameni prin sisteme computaționale bazate pe reguli.

Următoarea eră este Machine Learning Era (1980-2010), ce a marcat un punct de cotitură al progresului, marcând trecerea de la algoritmi "hard-coded logic" către sisteme de învățare pe baza datelor. Deși această tehnologie rămânea dependentă de intervenția umană, ea a introdus modele statistice de învățare automată, precum Support Vector Machines (SVM) și arbori de decizie, tehnici cărora le-a fost demonstrată utilitatea în aplicații de clasificare și recomandare. Această etapă a reprezentat un stadiu de tranziție față de paradigma simbolică, însă era caracterizată de un nivel redus de automatizare.

Era Deep Learning (2010–prezent): Această etapă, aflată în continuă evoluție, a fost impulsionată de creșterea exponențială a puterii de calcul și de disponibilitatea unor volume masive de date (Big Data). Prin intermediul rețelelor neuronale profunde, inclusiv al celor convoluționale (CNN), sistemele actuale posedă capacitatea de a extrage și de a învăța în mod autonom reprezentări ierarhice direct din date brute (primitive). Această paradigmă a revoluționat procesarea limbajului natural, recunoașterea formelor și a semnalelor vocale, depășind pragurile tradiționale de percepție artificială și atingând, în anumite domenii, performanțe comparabile cu cele umane.

Pe această bază a început era AI-ului generativ (2014–prezent), cu introducerea arhitecturii Transformer [VSP⁺17] în 2017, ce a permis dezvoltarea de modele precum Generative Pre-trained Transformer (GPT) [YRS⁺24]. Aceste sisteme s-au mutat de la instrucțiuni la generare, producând text, cod și fișiere media coerente. A fost contextul necesar pentru a face inteligența agentică fezabilă.

În cele din urmă, s-a ajuns la era agenților (2022–prezent), considerată frontiera actuală a dezvoltării inteligenței artificiale, în care capacitățile generative ale modelelor de tip LLM (Large Language Models) sunt valorificate pentru a susține acțiunea autonomă și luarea deciziilor. Din ce în ce mai mult, acești agenți evoluează în sisteme multi-agent, exemplificate de framework-uri precum CrewAI [TD25] și AutoGen [PMEM⁺23], unde rolurile specializate și colaborarea orchestrată permit echipelor de agenți să abordeze probleme complexe. Spre deosebire de deliberarea algoritmică a paradigmei simbolice, această etapă este definită de paradigma neurală, în care agentivitatea rezultă din orchestrarea stocastică a modelelor generative.

Progresul cronologic aduce context esențial, Agentul Intelligent nu este un simplu progres liniar, ci o arhitectură construită pe nivelele fundațiilor celor precedente. AI-ul agentic este vârful progresiei, definit ca o entitate ce poate percepe medii, raționamentul critic al scenariilor și acțiunea proactivă pentru obținerea obiectivelor predefinite [AADC25]. Comparativ cu generațiile anterioare de AI, care necesită instrucțiuni pas cu pas, Agen-

tul lucrează cu un grad de independență, adaptându-se la condiții flexibile și colaborarea cu alți agenți sau unelte de lucru pentru a ajunge la output-ul dorit de utilizatori.

3.2 Agentul Intelligent

Inteligența artificială agentică (Agentic AI) înglobează clasa sistemelor autonome capabile să execute seturi de sarcini complexe pe perioade îndelungate de timp, fără supervizare umană directă. Această paradigmă se remarcă prin abilitatea de a asimila contextul și de a fundamenta decizii de sine stătătoare. Astfel de sisteme sunt proiectate să opereze cu un grad ridicat de autonomie, ceea ce le permite să navigheze prin medii dinamice, să gestioneze situații imprevizibile și să își optimizeze performanța în mod continuu [AAD25].

Pentru a putea înțelege capacitățile unui Agent Intelligent, este nevoie să înțelegem toate aspectele tehnice ale sistemelor și arhitecturilor agentice [AKD25], prin comparație cu iterațiile anterioare de agenți.

3.2.1 Comparații cu Agenți Clasici

În timp ce sistemele de agenți clasici, sau Inteligență Artificială clasică, sunt construite pe seturi de reguli și modele supervizate create pentru task-uri specifice, Inteligența Agentică integrează autonomia și adaptare, având o gamă mai largă de funcționalități. Putem prezenta diferențele în următoarele exemple puse și în tabelul 3.2.1:

1. **Agenții clasici.** Acești agenți excelează în medii controlate. Sunt foarte puternici cu date ce urmează un flux constant, dar sunt volatili la schimbări bruște;
2. **Agentul Intelligent (Agentic AI).** Comparativ cu cel simplu, acesta își schimbă strategiile în timp real cu datele, trenduri și schimbări neașteptate, facându-l mult mai rezistent și adaptiv;
3. **Reinforcement Learning.** Acesta strategie este excelentă pentru optimizarea deciziilor în medii dinamice, învățând prin încercare și eroare să maximizeze o recompensă pe termen lung.

Feature	Agenții Clasici	Agenții RL	Agentul Inteligent
Scopul Principal	Automatizarea unei sarcini fixe	Optimizare bazată pe recompensă	Autonomie orientată spre obiective
Adaptabilitate	Scăzută	Moderată	Ridicată
Tipul de învățare	Supravegheată	Învățare prin recompensă	Hibridă, incluzând RAG și Memorie
Aplicații	Sisteme statice	Medii dinamice	Sarcini complexe cu obiective multiple

Tabela 3.1: Comparația între agenții clasici, agenții RL și Agentic AI [AKD25]

3.2.2 Fundamente tehnologice

Dezvoltarea Agenților Inteligenți sunt dependenți în principal de algoritmi și framework-uri pentru obiective, context adaptabil și decizii automate.

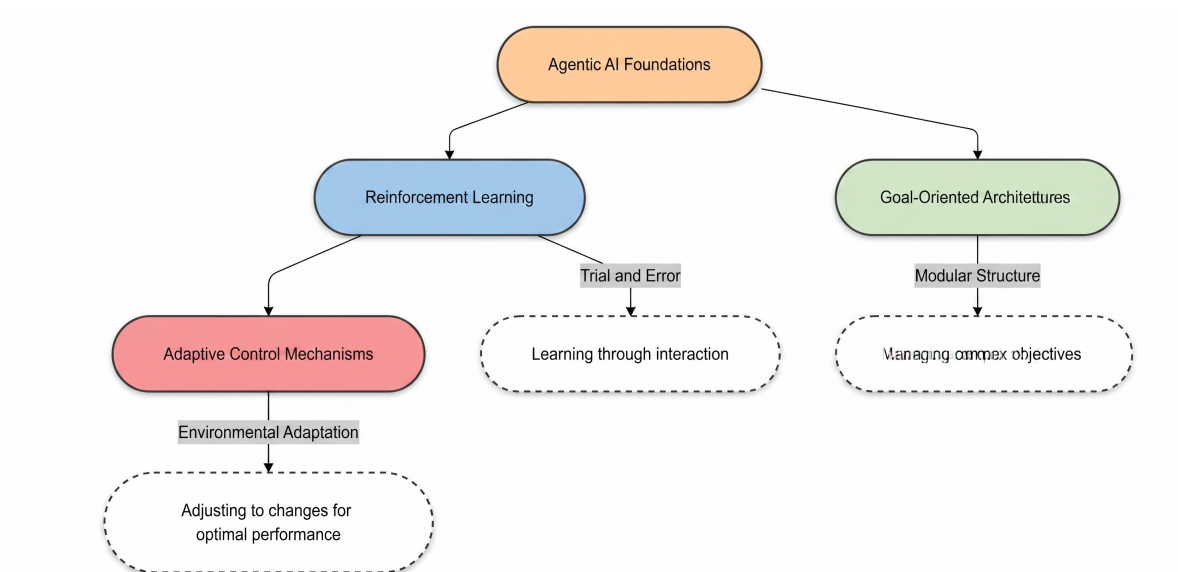


Figura 3.2: Ilustrare a componentelor Inteligenței Agentice [AKD25]

Figura 3.2 oferă o reprezentare vizuală a acestor cadre fundamentale, ilustrând modul în care acestea interacționează pentru a permite un comportament autonom, adaptabil și orientat către obiective în cadrul sistemelor de Inteligența Agentică (Agentic AI). Împreună, aceste fundamente tehnice dotează Agentic AI cu capacitățile structurale și funcționale necesare pentru a gestiona sarcini complexe și evolutive în mod independent, distingându-se de sistemele AI tradiționale care se bazează pe parametri și instrucțiuni rigid definite.

Reinforcement Learning (RL) este centrul multor sisteme de AI cu abilitatea de a

învăța din încercări și greșeli. În RL, agenți sunt atreanați să maximizeze recompensele cumulative prin interacțiunea cu mediul de dezvoltare, adaptându-și acțiunile pentru a ajunge la un specific scop în timp. Această paradigmă este deosebit de folositoare în Inteligența Agentică deoarece îi permite sistemului să își rafineze strategiile bazate pe feedback. Cum este reprezentat în figura 3.2, RL suportă învățarea prin iterații pentru optimizarea în timp.

Arhitectura Orientată pe Obiectiv (Goal-Oriented Architecture) furnizează un cadru structural pentru organizarea sarcinilor complexe în sistemele agentice, ce prioritizează mai multe obiective în același timp. Aceste arhitecturi au o structură modulară, unde rutinele sunt împărțite în mai multe subrutine pentru optimizarea paralelizării agentului. După cum este atașat în figura 3.2, Arhitectura orientată pe obiectiv facilitează "Organizarea obiectivelor complexe (Managing complex objectives)", lăsând agenții să se apuce de sarcini în pași structurați.

Mecanisme Adaptive de Control (Adaptive Control Mechanisms) asigură că Inteligența Agentică se poate ajusta asupra unui mediu interschimbabil. Când introducem acest mecanism, agenți recalibrează parametri ca răspuns la variațiile externe, precum schimbările datelor sau a perturbărilor neprevăzute. Tehnicile precum "meta-learning-ul", în care agenții învață să se adapteze pe baza experiențelor anterioare, permit o reziliență și o flexibilitate sporite. După cum este prezentat în diagrama de flux 3.2, mecanismele de control adaptiv asigură adaptarea la mediu, permițând agenților să mențină o performanță optimă chiar și în condiții schimbătoare.

3.2.3 Capabilitățile Agentului

Recente dezvoltări în Agentic AI s-au axat pe capabilitățile cheie ale structurii agenților moderni. Printre acestea se numără raționamentul și planificarea, care ajută agentul să anticipeze rezultate, să prioritizeze sarcini și să se adapteze la strategii dinamice, fiind critice pentru organizarea complexă a sarcinilor multi-obiectiv aflate în mediile de dezvoltare; utilizarea uneltelor și integrarea, prin care agenții sunt echipați cu abilitatea de a folosi unelte externe și API-uri ce pot executa o gamă largă de operațiuni digitale; mecanismele de memorie, în care modele semantice și episodice permit sistemului Agentului Inteligent să rețină informații contextuale, îmbunătățindu-i abilitatea de a reapela interacțiuni anterioare și ajutând la optimizarea sarcinilor viitoare; Retrieval-Augmented Generation (RAG), care permite agentului să extragă dinamic informații din cunoștințe externe, îmbunătățind astfel puterea pe care o are contextul în obținerea rezultatelor, fiind deosebit de important în conversațiile cu agenți sau în luările de decizii în timp real; și, în final, ajustarea fină pe instrucțiuni (*instruction tuning*), proces care asigură capabilitatea agentului de a înțelege și executa sarcini specifice.

3.3 Aplicații ale Inteligenței Agentice

3.3.1 Aplicații în Dezvoltarea Software

În dezvoltarea software, Agentul AI operează într-un ritm de inovație rapidă și cu mai puțin efort uman. Agenții asistă în ciclul de Software Development, începând de la cerințele de intrare până la gazduirea aplicației. În timpul codării, agenții pot genera, face debugging, refactorizare de cod, totul pe baza limbajului natural și integrării unui sistem de versionare ce colaborează cu dezvoltatorii umani [AADC25].

În domeniul automatizării proceselor (*workflow automation*), aceste elemente converg pentru a crea sisteme reziliente. Spre deosebire de automatizarea tradițională, care este rigidă în fața schimbărilor, un agent poate identifica un blocaj într-un flux de management de proiect prin analiza dependențelor dintre sarcini și a disponibilității echipei [IN24]. Ulterior, acesta poate propune realocări alternative de resurse și poate acționa prin reierarhizarea priorităților sau prin notificarea factorilor de decizie. Această sofisticare este susținută de bucle de feedback care permit ajustarea dinamică a comportamentului sistemului.

3.3.2 Aplicații în Procecele Business

Inteligența Agentică (Agentic AI) are un impact profund asupra proceselor de afaceri prin automatizarea operațiunilor end-to-end cu o inteligență fără precedent. În domeniul achizițiilor, agenții pot negocia cu furnizorii prin analizarea tendințelor pieței, evaluarea ofertelor și finalizarea contractelor în mod autonom, reducând durata ciclurilor de la săptămâni la ore [BJH⁺22]. Această aplicație valorifică integrarea datelor în timp real, prin care agenții extrag informații din baze de date globale pentru a lua decizii informate, asigurând economii de costuri și conformitate.

În managementul resurselor umane, Agentic AI eficientizează procesul de recrutare prin trierea CV-urilor, programarea interviurilor și chiar realizarea evaluărilor preliminare prin interfețe conversaționale [KR25]. Aceste sisteme vor încorpora algoritmi de detectare a prejudecăților pentru a promova angajarea echitabilă, adaptându-se în același timp la cerințele profesionale în continuă schimbare, pe baza evoluțiilor din industrie. Operațiunile financiare beneficiază în mod similar, agenții automatizând reconcilierea, detectarea fraudelor și prognozele prin verificarea încrucișată a tranzacțiilor cu modelele istorice și indicatorii economici externi [SEA26].

Fluxurile de lucru în marketing sunt transformate prin gestionarea personalizată a campaniilor, unde agenții analizează datele clienților pentru a proiecta, implementa și optimiza strategii în timp real. De exemplu, un agent poate detecta reclamele cu performanțe scăzute, poate efectua teste A/B pentru alternative și poate alocă bugetele în mod dinamic [CL25]. În cadrul tuturor acestor aplicații, elementul comun este

agilitatea sporită, permițând companiilor să răspundă la volatilitatea pieței cu o latență minimă, stimulând în cele din urmă creșterea veniturilor și reziliența operațională [McA24].

3.3.3 Aplicații Industriale

Inteligența Artificială Agentică are potențialul de a revoluționa numeroase industrii, inclusiv sănătatea, finanțele, educația și producția. În sănătate, de exemplu, Agentic AI poate fi utilizat pentru a analiza datele pacienților, a identifica tipare anormale și a alerta personalul medical corespunzător în caz de deteriorare a stării de sănătate [BBA25]. Dispozitivele bazate pe AI ar putea, de exemplu, să prevină întârzierea diagnosticelor vitale prin monitorizarea indicatorilor de sănătate cheie ai pacienților și notificarea acestora atunci când situația se agravează. În finanțe, algoritmi de Agentic AI pot asista în efectuarea tranzacțiilor de investiții, detectarea activităților frauduloase și furnizarea de soluții de investiții personalizate [FYJ⁺25]. Aceștia sunt capabili să evalueze situația de pe bursă, luând decizii independente privind achiziția sau vânzarea de titluri de valoare și ajustându-și strategiile la condițiile schimbătoare în timp real, ceea ce îmbunătățește semnificativ calitatea actului managerial și reduce necesitatea intervenției umane.

3.4 Scalabilitate

Cercetările curente utilizează un număr limitat de agenți, fiind depuse puține eforturi pentru a scala numărul acestora în vederea creării unor sisteme mai complexe. Creșterea numărului de agenți poate conduce la o specializare mai accentuată și la îndeplinirea unor sarcini mai ample și mai complexe, îmbunătățind astfel eficiența proceselor, precum cele din dezvoltarea software sau formularea politicilor guvernamentale. Totodată, scalarea permite descoperirea vulnerabilităților și a potențialelor riscuri ce pot apărea în cadrul sistemelor de agenți la scară largă.

O modalitate intuitivă și simplă de a mări numărul de agenți constă în predeterminarea acestuia de către proiectant [Gar26]. Mai precis, prin stabilirea prealabilă a numărului de agenți, a rolurilor și atributelor respective, a mediului de operare și a obiectivelor, proiectanții pot permite agenților să interacționeze, să colaboreze sau să se angajeze în alte activități în mod autonom pentru a atinge scopurile comune. Totuși, această abordare statică devine limitativă pe măsură ce sarcinile devin mai complexe sau când diversitatea participanților sociali crește, necesitând un număr mai mare de agenți pentru atingerea obiectivelor dorite. În astfel de cazuri, sistemul trebuie re-proiectat manual și repornit de către proiectant.

O altă abordare viabilă pentru scalarea numărului de agenți este prin ajustări

dinamice [IN24]. Aceasta este implementată de către agenții înșiși, care pot delega sarcini în mod autonom altor agenți, distribuind astfel volumul de muncă, reducând propria sarcină și atingând obiectivele comune mai eficient. În această abordare, proiectantul definește doar cadrul inițial, acordând agenților o autonomie și o capacitate de auto-organizare sporite, ceea ce conferă sistemului un caracter mai autonom. Agenții își pot gestiona mai bine volumul de muncă în condiții și cerințe aflate în continuă evoluție, oferind o flexibilitate și o scalabilitate superioare.

Deși creșterea numărului de agenți poate conduce la îmbunătățirea eficienței sarcinilor, există mai multe provocări iminente. De exemplu, povara computațională va crește odată cu numărul mare de agenți inteligenți utilizați, necesitând un design arhitectural mai bun și optimizare computațională pentru a asigura funcționarea fluidă a întregului sistem.

Pe măsură ce numărul de agenți crește, provocările legate de comunicare și propagarea mesajelor devin considerabile, deoarece rețeaua de comunicare a întregului sistem devine extrem de complexă. Un sistem cu mai mulți agenți ar putea amplifica acest risc, făcând comunicarea și schimbul de informații mai puțin fiabile. Mai mult, dificultatea coordonării agenților crește proporțional cu numărul acestora, putând face cooperarea mai dificilă și mai puțin eficientă, ceea ce poate impacta progresul către atingerea obiectivelor comune.

3.5 Încrederea și Fiabilitatea

Asigurarea încrederii a rămas în mod constant o problemă critică mai ales în domeniul învățării profunde (*deep learning*). Deși Agentic AI a atras o atenție semnificativă datorită performanțelor remarcabile în diverse sarcini, natura lor de tip "cutie neagră" (*black-box*) a mascat factorii fundamentali care stau la baza acestor performanțe superioare [XCG⁺25].

În plus, prejudecățile (*biases*) inerente din datele de antrenament pot infiltra rețelele neuronale. De exemplu, modelele de limbaj părtinitoare ar putea genera discursuri ce implică discriminare rasială sau de gen, care ar putea fi amplificate în cadrul aplicațiilor agentice, dând impact advers într-un context social [CSM⁺23]. Totodată, modelele de limbaj sunt afectate de probleme severe de halucinație, fiind predispuse să producă texte care deviază de la faptele reale, subminând astfel credibilitatea agenților bazați pe LLM.

3.5.1 Robustețea Adversarială

Robustețea adversarială reprezintă un factor determinant în aplicabilitatea sistemelor de învățare profundă, fiind intens studiată în domenii precum viziunea artificială,

procesarea limbajului natural și învățarea prin consolidare (*reinforcement learning*). În esență, un sistem robust este acela care, atunci când este confruntat cu o intrare perturbată ($x' = x + \delta$), reușește să producă rezultatul original (y), în timp ce un sistem vulnerabil este indus în eroare, generând un rezultat inconsistent (y').

Cercetătorii au constatat că modelele de limbaj pre-antrenate sunt deosebit de susceptibile la atacuri adversariale, ceea ce conduce la generarea de răspunsuri eronate. Acest fenomen este observat pe scară largă chiar și în cazul modelelor de limbaj de mari dimensiuni (LLM), ridicând provocări semnificative pentru dezvoltarea agenților bazați pe LLM. Există, de asemenea, metode de atac relevante, cum ar fi otrăvirea seturilor de date (*dataset poisoning*) [GDGG17], atacurile de tip "backdoor" [CSC⁺21] și atacurile specifice prompt-urilor [SLY⁺22], care au potențialul de a determina LLM-urile să genereze conținut toxic.

În timp ce impactul atacurilor adversariale asupra LLM-urilor este limitat la erori textuale, în cazul agenților bazați pe LLM, care dispun de o gamă mai largă de acțiuni, atacurile adversariale ar putea, în mod potențial, să îi determine pe aceștia să întreprindă acțiuni cu adevărat distructive, rezultând prejudicii sociale substanțiale. În ceea ce privește modulul de percepție al agenților bazați pe LLM, dacă acesta primește intrări adversariale din alte modalități, cum ar fi imagini sau semnale audio, agenții pot fi, de asemenea, induși în eroare, ceea ce conduce la rezultate incorecte sau distructive. În mod similar, modulul de acțiune poate fi, la rândul său, vizat de atacuri adversariale. De exemplu, instrucțiunile modificate malițios, axate pe utilizarea uneltelor, ar putea determina agenții să execute manevre eronate [QHL⁺24].

Pentru a aborda aceste probleme, cercetătorii pot utiliza tehnici tradiționale, cum ar fi antrenamentul adversarial (*adversarial training*), augmentarea datelor adversariale și detecția eșantioanelor adversariale, pentru a spori robustețea agenților bazați pe LLM. Totuși, elaborarea unei strategii care să abordeze în mod holistic robustețea tuturor modulelor din cadrul agenților, menținând în același timp utilitatea acestora fără a compromite eficacitatea, reprezintă o provocare mult mai formidabilă. În plus, poate fi utilizată o abordare de tip "human-in-the-loop" pentru a supraveghea și a oferi feedback cu privire la comportamentul agenților.

3.5.2 Construirea unui Agent Onest

Construirea unui agent inteligent care să fie onest și demn de încredere este o cerință urgentă. Eforturi recente de cercetare se concentrează pe ghidarea modelelor pentru a expune procese de gândire sau explicații în timpul etapei de inferență, cu scopul de a spori credibilitatea predicțiilor [TMPB23].

În plus, integrarea bazelor de cunoștințe externe și a bazelor de date poate atenua problemele legate de halucinații. De exemplu, tehnici precum supravegherea procesului

(*process supervision*) pot face ca rezultatele inferenței și raționamentul LLM-urilor să fie mai fiabile. Spre exemplu, în cadrul proiectului MedAgents [TZZ⁺24], autorii îmbunătățesc capacitatea de raționament a modelelor în domeniul medical prin jocuri de rol și interacțiuni pe mai multe rânduri, generând răspunsuri mult mai sigure. Mai mult, utilizarea metodelor de eliminare a prejudecăților (*debiasing*) și a tehnicilor de calibrare poate contribui la atenuarea potențialelor probleme de echitate din cadrul LLM-urilor.

3.5.3 Alte riscuri potențiale

Utilizarea abuzivă (Misuse): Agenții bazați pe LLM au fost dotați cu capacități extinse și complexe, permițându-le să îndeplinească o gamă largă de sarcini. Totuși, pentru persoanele cu intenții malițioase, acești agenți pot deveni instrumente care reprezintă amenințări la adresa societății. Aceștia ar putea fi exploatați pentru a manipula opinia publică, a disemina informații false, a compromite securitatea cibernetică sau a organiza activități frauduloase. Prin urmare, înainte de implementare, sunt necesare politici de reglementare stricte și un design de securitate robust.

Șomajul: Pe măsură ce agenții autonomi avansează, aceștia capătă capacitatea de a asista oamenii în diverse domenii (scriere de cod, depanare, completarea formularelor), atenuând presiunea muncii. Totuși, această dezvoltare ridică temeri legate de înlocuirea locurilor de muncă umane, generând o criză socială. Este nevoie de măsuri educaționale și politici de sprijin pentru a asigura o plasă de siguranță în timpul acestei tranziții.

Amenințarea la adresa bunăstării rasei umane: Pe măsură ce agenții evoluează, oamenii (inclusiv dezvoltatorii) ar putea întâmpina dificultăți în a-i înțelege, prezice sau controla în mod fiabil. Dacă acești agenți depășesc inteligența umană și dezvoltă propriile „ambiții”, ar putea apărea riscuri existențiale. Viitorul necesită o înțelegere profundă a mecanismelor de operare ale acestor agenți și anticiparea impactului lor direct sau indirect. Evol

Capitolul 4

Abordări Anterioare în Industrie

Înainte de a detalia soluția propusă, este necesară o trecere în revistă a abordărilor existente în industrie pentru gestionarea și prelucrarea fișierelor asistată de inteligență artificială. Capitolul de față analizează patru categorii de soluții relevante pentru proiectul FileO: asistenții de sistem de operare cu integrare AI (Microsoft Copilot), agenții generalişti de tip "desktop coworker" (Claude Cowork), utilitarele specializate de analiză a spațiului pe disc (WizTree) și platformele web de prelucrare a fișierelor. Secțiunea finală din cadrul acestui capitol compară aceste abordări cu soluția propusă.

4.1 Microsoft Copilot

Microsoft integrează asistentul Copilot direct în Windows 11, inclusiv în File Explorer. Cele mai recente build-uri Windows Insider introduc o opțiune în meniul contextual care permite utilizatorilor să interacționeze cu fișierele prin acțiuni bazate pe AI, precum sumarizarea conținutului, eliminarea fundalului din imagini și ștergerea obiectelor nedorite dintr-o fotografie [Tec26]. Pentru fișierele imagine (.jpg, .jpeg, .png), Copilot oferă opțiuni precum estomparea fundalului, eliminarea fundalului și ștergerea obiectelor, direct din meniul contextual al File Explorer.

Microsoft 365 Copilot extinde aceste capabilități la nivelul fișierelor stocate pe OneDrive: utilizatorii pot sumariza documente, genera FAQ-uri, compara fișiere și pune întrebări despre conținutul acestora direct din File Explorer și din OneDrive Activity Center [Adm26]. Planurile anunțate de Microsoft includ extinderea suportului dincolo de imagini, către documente Office, foi de calcul și prezentări, precum și operații de tip batch exprimate în limbaj natural (de exemplu, redenumirea în masă a unor facturi după un format de dată specific) [gTN26].

La nivel enterprise, Microsoft oferă prin Copilot Studio agenți de tip *computer-use*, capabili să execute fluxuri multi-aplicație (formulare, foi de calcul Excel, ecrane legacy), cu mecanisme de *human-in-the-loop*: agentul poate insera puncte de aprobare explicite înainte de a executa acțiuni sensibile, fluxul rămânând suspendat până la confirmarea

unui operator uman [Com26].

Limitările acestei abordări, din perspectiva proiectului de față, sunt legate de dependența de ecosistemul Microsoft 365 și de cloud (multe funcții necesită un abonament activ și o conexiune la serviciile Microsoft), de scopul restrâns al acțiunilor disponibile per fișier (un set fix de operații predefinite, fără un mecanism general de apelare al instrumentelor și de absența unui set unificat de instrumente de conversie media (video, audio, subtitrări, modele 3D) similar celui din FileO.

4.2 Claude Cowork

Claude Cowork este un mod al aplicației desktop Claude, lansat de Anthropic, care transformă modelul de limbaj într-un "coleg de muncă digital" capabil să execute sarcini complexe, multi-pas, asupra fișierelor locale [Ant26]. Spre deosebire de o interfață de chat clasică, în care modelul oferă doar răspunsuri statice, agentul Cowork formulează un plan, execută pașii acestuia, se auto-corectează și solicită clarificări atunci când este necesar, permițând rezolvarea unor sarcini complexe cu supraveghere minimă [Ven26].

Din punct de vedere al gestionării fișierelor, utilizatorul desemnează un folder local la care agentul are acces; în interiorul acestui "sandbox", Claude poate citi fișiere existente, le poate modifica sau poate crea fișiere noi [Ram26]. Exemple tipice de utilizare oferite de Anthropic includ reorganizarea unui folder de descărcări dezordonat (sortare și redenumire inteligentă a fiecărui fișier), generarea unei foi de calcul cu cheltuieli pornind de la o colecție de capturi de ecran cu bonuri fiscale, sau redactarea unui raport pe baza unor notițe împrăștiate în mai multe documente [Mar26]. Cowork poate reutiliza și conectorii existenți ai Claude (Asana, Notion, PayPal etc.), extinzând acțiunile dincolo de sistemul de fișiere local.

Caracterul general al agentului este atât principalul punct forte, cât și principala limitare din perspectiva proiectului de față: Cowork nu dispune de o bibliotecă fixă de tool-uri specializate de procesare media (nu are integrate modele proprii de super-rezoluție, eliminare fundal, transcriere audio sau generare de subtitrări), ci rezolvă astfel de cerințe fie prin raționament și scriptare ad-hoc, fie prin apeluri către servicii externe conectate. În plus, execuția se bazează integral pe un model de limbaj rulat în cloud, fără o componentă locală dedicată procesării media, așa cum este gateway-ul AI din FileO (Swin2SR, Whisper, Groq).

4.3 WizTree

WizTree este o unealtă Windows specializată în analiza rapidă a spațiului ocupat pe disc [Wizb]. Spre deosebire de utilitarele clasice care parcurg recursiv sistemul de fișiere și interoghează fiecare fișier în parte pentru a-i determina dimensiunea, WizTree citește

direct *Master File Table* (MFT) a partițiilor formate NTFS, o structură care conține numele, dimensiunea și locația fiecărui fișier de pe disc. Prin citirea directă a MFT, ocolind complet API-ul standard al sistemului de operare, WizTree obține o creștere semnificativă de performanță, scanând discuri întregi în câteva secunde, comparativ cu minute pentru aplicații similare [Pur26].

Rezultatele scanării sunt afișate printr-o hartă vizuală de tip *treemap*, în care dimensiunea fiecărui dreptunghi este proporțională cu spațiul ocupat de fișierul sau folderul corespunzător, permițând identificarea rapidă a celor mai mari consumatori de spațiu [Wiza]. WizTree oferă și o listă detaliată (*File View*) a tuturor fișierelor și folderelor în ordinea descrescătoare a dimensiunii, precum și o funcție de detectare a fișierelor duplicate, bazată pe nume, dimensiune și, opțional, dată. Aplicația gestionează corect fișierele cu legături hard (*hard links*), evitând numărarea repetată a spațiului ocupat de acestea, iar spațiul total raportat corespunde celui afișat de Windows.

Din perspectiva proiectului de față, WizTree reprezintă un exemplu reprezentativ de unealtă *single-purpose*: extrem de eficientă pentru sarcina sa specifică, dar limitată strict la analiza și vizualizarea spațiului pe disc, fără capabilități de conversie, fără asistent AI și fără posibilitatea de a acționa asupra fișierelor identificate (ștergere, mutare, conversie) altfel decât prin deschiderea lor manuală în Explorer. De asemenea, dependența de MFT limitează aplicația la sisteme de fișiere NTFS pe Windows.

4.4 Site-uri de Lucru cu Fișiere

O categorie larg utilizată de soluții pentru prelucrarea fișierelor o reprezintă platformele web de conversie și editare, accesibile fără instalare locală. CloudConvert este una dintre cele mai cunoscute platforme de acest tip, suportând conversia între peste 200 de formate de fișiere (documente, imagini, audio, video, arhive), îmbinare de fișiere, generare de capturi de ecran ale paginilor web și adăugare de watermark-uri cloudconvert. Platforma este certificată ISO 27001, criptează fișierele prin SSL și le șterge automat de pe servere în termen de 24 de ore [One]. Smallpdf oferă funcționalități similare axate pe formatul PDF, conversie bidirecțională între PDF și Word, Excel, PowerPoint sau imagini, direct din browser, fără cont, fișierele fiind șterse de pe servere după o oră [Sma]. remove.bg este specializat în eliminarea automată a fundalului din imagini folosind modele de segmentare bazate pe AI, oferind un rezultat similar cu tool-ul de eliminare a fundalului din FileO, dar exclusiv ca serviciu web.

Aceste platforme au avantajul accesibilității universale (orice browser, fără instalare) și al actualizării continue a modelelor AI utilizate, însă prezintă limitări importante pentru un flux de lucru intensiv: fiecare operație necesită încărcarea fișierelor pe serverele unui furnizor extern, ceea ce ridică probleme de confidențialitate pentru documente sensibile; operațiile sunt în general unitare (un fișier sau un lot, o singură transformare),

fără posibilitatea de a înlănțui automat mai mulți pași într-un flux complex; iar limitele de dimensiune, numărul de conversii gratuite și viteza de upload/download depind de planul de abonament. Nu există, în niciuna dintre aceste platforme, o componentă de tip agent conversațional capabilă să orchestreze mai multe instrumente pentru a duce la îndeplinire o sarcină formulată în limbaj natural.

4.5 Abordarea Soluției

Soluția propusă, FileO, se desprinde un tipar comun: fiecare abordare existentă acoperă bine fie componenta de *instrumente de prelucrare a fișierelor*, fie componenta de *agent AI conversațional*, dar nu pe ambele într-un singur produs, local și extensibil. Față de Microsoft Copilot, FileO oferă un set de instrumente mult mai larg și uniform (conversie de imagini, audio, video, documente și modele 3D, eliminare de fundal, generare de subtitrări, îmbinare PDF, creare de unități virtuale și analiză de spațiu), expuse agentului printr-un contract comun (DEFINITION JSON Schema) și fără dependență de ecosistemul Microsoft 365 sau de un abonament cloud. Față de Claude Cowork, agentul din FileO renunță la flexibilitatea unui agent generalist în favoarea unei biblioteci fixe și extensibile de tool-uri specializate, validate și testate individual (proces descris în 5), iar arhitectura Plan-and-Execute, cu pași de tip `tool` și `llm` executați secvențial și sintetizați într-un răspuns final, este conceptual înrudită cu modul în care Cowork "formulează un plan, execută pașii și se auto-corectează", dar aplicată unui domeniu restrâns și optimizat. Față de WizTree, modulul **SpaceAnalyzer** oferă o funcționalitate similară de identificare a fișierelor și folderelor mari, dar integrată ca un tool printre celelalte și accesibilă prin agent, fără a depinde de citirea directă a MFT, deși WizTree rămâne superior ca viteză brută de scanare pe volume foarte mari. Față de site-urile de lucru cu fișiere (CloudConvert, Smallpdf, remove.bg), avantajul central al FileO este procesarea *locală* a fișierelor, fără a le încărca pe infrastructura unui furnizor extern, eliminând atât problema confidențialității, cât și limitele impuse de planurile gratuite, și permițând înlănțuirea automată a mai multor operații printr-o singură cerere în limbaj natural.

În concluzie, niciuna dintre soluțiile analizate nu acoperă simultan cele trei dimensiuni pe care se bazează FileO: o bibliotecă unificată și extensibilă de instrumente de prelucrare media și fișiere, un agent conversațional cu planificare multi-pas și aprobare umană pentru acțiuni sensibile, și execuție locală/self-hosted fără dependență de servicii cloud externe pentru operațiile de bază. Poziționarea FileO la intersecția acestor trei dimensiuni reprezintă principala sa contribuție față de abordările anterioare din industrie.

Capitolul 5

FileO, Aplicație de Orchestrare a Fișierelor cu Ajutorul unui Agent Inteligent

5.1 Specificarea Aplicației

FileO este o aplicație desktop de tip orchestrator de fișiere, construită pe stiva Electron + React + TypeScript pentru interfața grafică și Flask (Python) pentru nucleul logic de pe mașina locală. Scopul principal este de a oferi utilizatorului un singur punct de control pentru operații diverse asupra fișierelor, conversii, analiză, îmbunătățire, transcripție, reorganizare pe disc, cu posibilitatea de a delega secvențe complexe unui agent de planificare bazat pe un model lingvistic de mari dimensiuni.

Specificarea aplicației descrie mai jos cerințele funcționale derivate direct din structura modulelor implementate și cerințele non-funcționale care guvernează calitățile de sistem.

5.1.1 Cerințe Funcționale

CF-1 Sistemul expune un catalog de unelte

Aplicația expune un catalog structurat de instrumente, grupate în mai multe categorii, detaliate în tabelul 5.1. Fiecare categorie reunește instrumente care operează pe același tip de fișier sau pe același domeniu funcțional. Unele instrumente utilizează modele AI locale (*usesAI = true*), în timp ce altele aplică exclusiv biblioteci deterministe.

Tabela 5.1: Categoriile de instrumente ale aplicației FileO

Cheie	Etichetă	Domeniu
<code>image</code>	Image	Procesare imagini raster și vectoriale
<code>audio</code>	Audio	Conversie și transcriere fișiere audio
<code>video</code>	Video	Compresie, conversie și analiză video
<code>documents</code>	Documents & Archives	Procesare Documente, PDF-uri și arhive
<code>3d</code>	3D & Modeling	Modele 3D, scene și fișiere CAD
<code>computer_tools</code>	Computer Tools	Gestionare disc, drive virtual și operații de folder

CF-2 Definirea structurată a parametrilor unui instrument

Pentru fiecare instrument din catalog, sistemul expune o definiție structurată a parametrilor necesari execuției: tipul fiecărui câmp (fișier, listă de fișiere, opțiune selectabilă, valoare numerică sau logică), caracterul obligatoriu, valorile implicite și opțiunile disponibile. Această definiție permite generarea automată, în interfața grafică, a formularului de configurare corespunzător, fără a necesita cunoștințe tehnice din partea utilizatorului. Ca referință, tabelul 5.1.1 detaliază câmpurile instrumentului *Image Converter* (`id: image-converter`), singurul descris exhaustiv în această secțiune; celelalte instrumente respectă aceeași structură.

Cheie câmp	Tip	Obligatoriu	Descriere
<code>inputFiles</code>	<code>multifile</code>	Da	Unul sau mai multe fișiere imagine de convertit
<code>outputFormat</code>	<code>select</code>	Da	Formatul țintă: <code>jpeg</code> , <code>png</code> , <code>webp</code> , <code>bmp</code> , <code>tiff</code> ; implicit <code>webp</code>
<code>quality</code>	<code>number</code>	Nu	Calitate compresie cu pierderi (1–100); se aplică doar la JPEG și WebP; implicit 85
<code>preserveMetadata</code>	<code>boolean</code>	Nu	Când este activ, metadatele EXIF sunt copiate în fișierul rezultat; implicit <code>true</code>

Tabela 5.2: Câmpurile instrumentului *Image Converter*

CF-3 Organizarea fișierelor în drive-uri virtuale

Sistemul permite utilizatorului să organizeze fișierele de pe disc în *drive-uri virtuale*: grupări logice de fișiere pe categorii, create fie prin scurtături către fișierele originale, fie prin mutarea fizică a acestora. Utilizatorul poate configura un folder de destinație implicit, în care instrumentele vor depune fișierele procesate, iar sistemul oferă operațiile de creare, listare și ștergere a drive-urilor virtuale.

CF-4 Accesul la funcționalitățile aplicației prin interfață programatică

Sistemul expune o interfață care permite gestionarea drive-urilor virtuale (creare, listare, ștergere și analiza spațiului ocupat pe disc), consultarea catalogului de instrumente și configurarea și execuția acestora, precum și inițierea și gestionarea sesiunilor cu agentul de planificare, trimiterea de sarcini, interogarea stării și configurarea agentului.

CF-5 Integrarea cu agentul de planificare

Aplicația permite utilizatorului să configureze conectarea la un server de inferență extern, prin care sunt redirecționate cererile către agentul de planificare. Utilizatorul poate descrie în limbaj natural o sarcină compusă (de ex. „transcrie toate fișierele audio din folderul X și mută-le în drive-ul Y”), iar agentul selectează și ordonează instrumentele din catalog pentru a o îndeplini.

CF-6 Analiza spațiului ocupat pe disc

Sistemul permite utilizatorului să analizeze rapid spațiul ocupat pe disc, prin citirea directă a structurii interne a sistemului de fișiere, fără a fi necesară parcurgerea recursivă a directoarelor.

5.1.2 Cerințe Non-Funcționale

CNF-1 Performanță

Operațiile de catalog și de listare a drive-urilor virtuale trebuie să returneze răspuns în mai puțin de 200 ms în condiții de rețea locală. Instrumentele care nu utilizează AI (spre exemplu: conversii format, fuzionare PDF, vizualizare obiecte 3d) nu trebuie să introducă latență perceptibilă pentru fișiere de până la 50 MB. Instrumentele bazate pe modele AI (transcriere audio, îmbunătățire imagine, eliminare fundal) operează asincron, frontend-ul afișează progresul și nu blochează interfața pe durata inferenței.

CNF-2 Rulare locală și confidențialitate

Toate procesările sunt efectuate pe mașina utilizatorului. Niciun fișier utilizator nu este transmis la servicii externe fără consimțământ explicit.

CNF-3 Compatibilitate

Aplicația este distribuită ca binar Electron și trebuie să ruleze pe Windows 10/11 pe arhitecturi x86-64.

CNF-4 Extensibilitate

Adăugarea unui instrument nou presupune exclusiv adăugarea unei înregistrări în `catalog.py` și implementarea handler-ului Python corespunzător. Modificarea frontend-ului nu este necesară deoarece interfața este generată dinamic din schema instrumentului.

CNF-5 Mentenabilitate

Codul backend este organizat în șabloane independente, fiecare cu responsabilitate unică (*drive*, *tools*, *agent*). Catalogul de instrumente este decuplat de logica de execuție, metadatele UI (câmpuri, pași de utilizare, etichete) sunt definite separat față de implementarea efectivă. Frontend-ul urmează structura React cu componente funcționale și tipizare TypeScript strictă.

CNF-6 Securitate locală

Serverul Flask ascultă exclusiv pe interfața de loopback (`127.0.0.1:5000`) și nu este expus în rețeaua locală sau externă. Cross-Origin Resource Sharing (CORS) este activat cu politică restrictivă pentru a permite exclusiv cererile din contextul aplicației Electron. Cheia API a agentului nu apare în jurnalele aplicației și nu este transmisă în răspunsurile REST returnate frontend-ului.

5.2 Proiectarea Aplicației

5.2.1 Arhitectura generală

FileO este structurată pe trei procese independente care comunică prin interfețe bine definite, prezentate în figura 5.1.

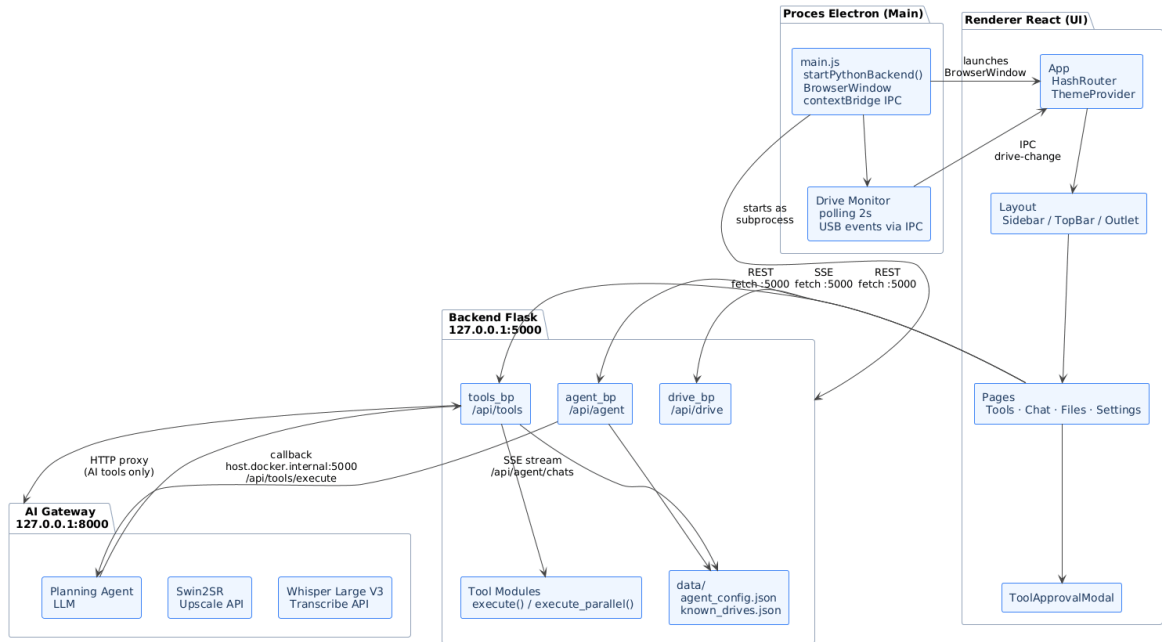


Figura 5.1: Arhitectura proceselor aplicației FileO

Procesul Electron pornește backend-ul Python la lansare (ca subprocess), creează fereastra grafică fără bordură nativă și expune un API IPC renderer-ului prin `contextBridge`, incluzând controlul ferestrei, dialoguri de selecție fișiere și notificări la conectarea/deconectarea drive-urilor fizice (polling la 2secunde).

Backend-ul Flask ascultă pe `127.0.0.1:5000` și organizează funcționalitatea în trei șabloane: `drive_bp` pentru gestionarea drive-urilor virtuale, `tools_bp` pentru catalogul și executorul de instrumente, și `agent_bp` pentru proxy-ul de comunicare cu agentul de planificare. Instrumentele care necesită confirmare umană sunt blocate pe un `threading.Event` până la aprobarea sau respingerea din interfața grafică.

Renderer-ul React folosește `HashRouter`, necesar deoarece Electron servește fișiere statice prin protocolul `file://`, și randează o interfață cu trei secțiuni principale: *My Drives*, *Agent* și *Tools*. Catalogul de instrumente este încărcat o singură dată la pornire de la `GET /api/tools/catalog` și utilizat pentru a genera dinamic paginile de detaliu și formularele de rulare, fără cod frontend hardcodat per instrument.

5.2.2 Diagrame de clase

Figura 5.2 ilustrează structura modulelor backend. Fiecare modul de instrument respectă un contract uniform: expune o constantă `DEFINITION` (dicționar cu schema JSON pentru agent) și o funcție `execute(input: dict) -> str` care returnează JSON serializat. Instrumentele cu volum mare expun suplimentar `execute_parallel()`. Șablonul `tools_bp` le agregă printr-un dicționar `_TOOLS` și le invocă polimorfic, adăugarea unui

instrument nou nu necesită modificarea logicii de rutare.

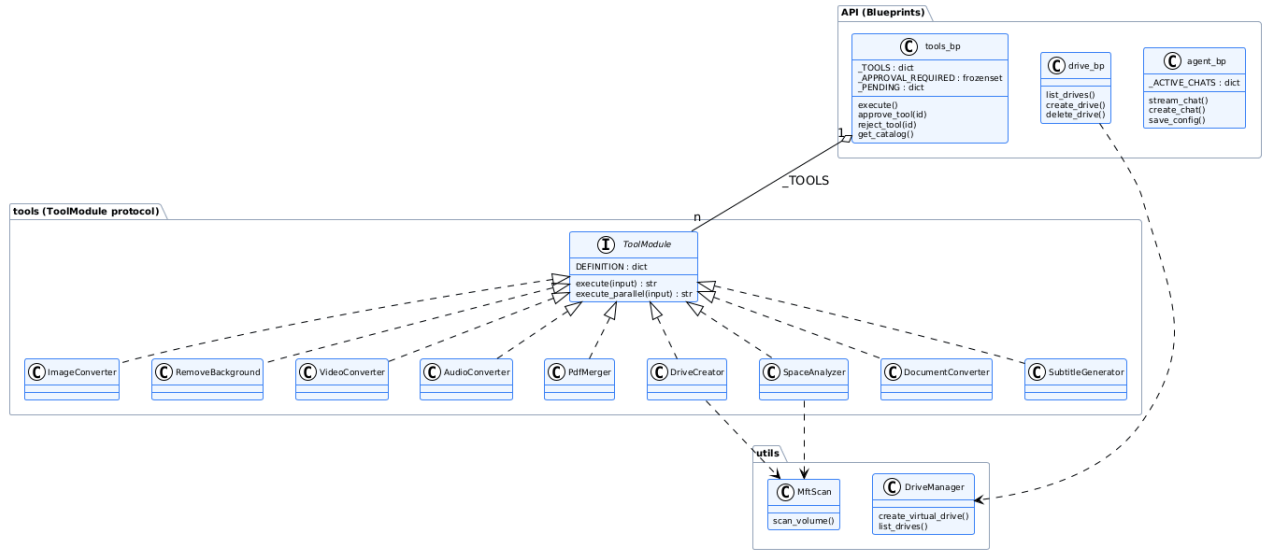


Figura 5.2: Structura modulelor de instrumente (backend)

Pe partea de frontend, figura 5.3 prezintă ierarhia componentelor React. App montează Layout (Sidebar + TopBar + Outlet), care gestionează navigarea. Fiecare instrument cu interfață proprie are o componentă XxxPage dedicată; instrumentele fără pagină personalizată sunt servite de componenta generică ToolDetail, care construiește formularul din schema catalogului primită de la backend.

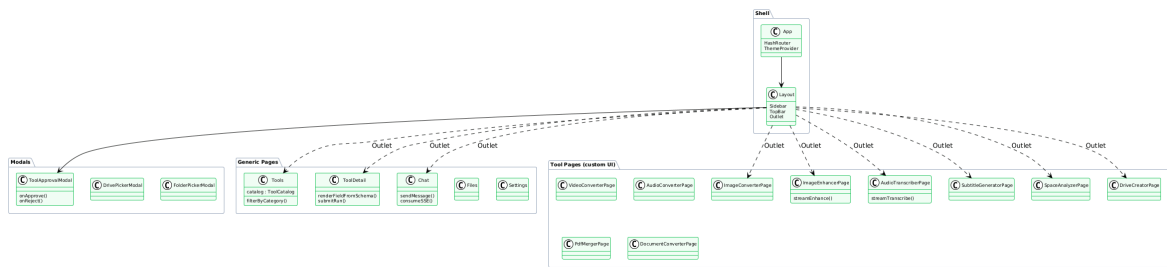


Figura 5.3: Ierarhia componentelor React (frontend)

5.2.3 Diagrame de secvență

Execuția unui instrument din interfața grafică

Figura 5.4 descrie fluxul pentru rularea unui instrument direct de către utilizator. Utilizatorul completează câmpurile de intrare a instrumentului și apasă *Run*; frontend-ul trimite un `POST /api/tools/<id>/run` cu parametrii serializați. Backend-ul apelează `execute()` pe modulul corespunzător, procesează fișierele și returnează un JSON cu

câmpurile **success**, **total**, **succeeded**, **failed** și **results**, unde **success** indică starea generală de reușită a operației, **total** reprezintă numărul total de fișiere identificate pentru procesare, **succeeded** și **failed** numărul de fișiere procesate cu succes, respectiv cu eroare, iar **results** conține o listă detaliată per fișier, cu calea acestuia și statusul individual de succes. Instrumentele AI (Image Enhancer, Audio Transcriber) urmează același flux, dar endpoint-ul **/stream** este apelat în locul **/run**, iar progresul este afișat în timp real prin Server-side Events (SSE).

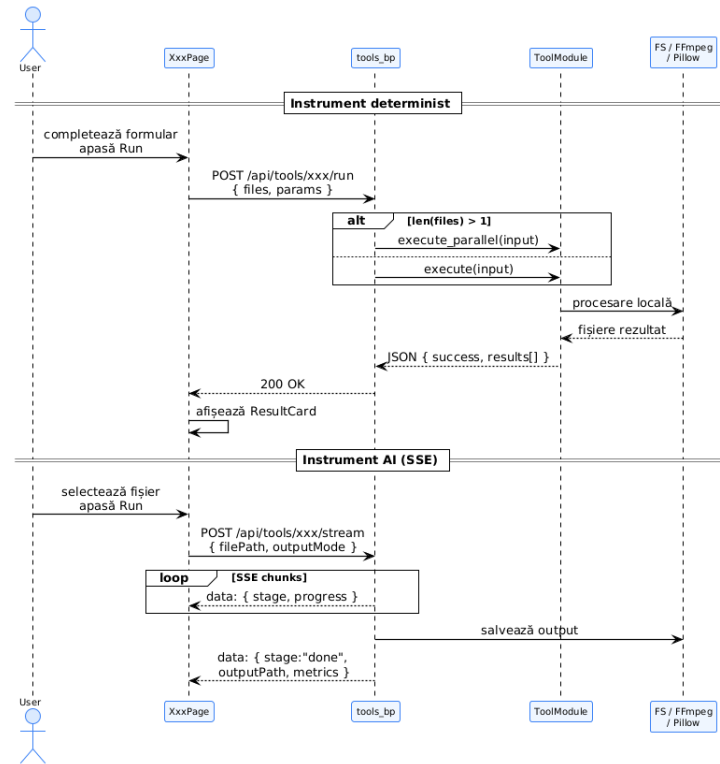


Figura 5.4: Secvența de execuție directă a unui instrument

Execuția unui instrument prin agent cu aprobare umană

Figura 5.5 prezintă fluxul mai complex în care agentul de planificare decide să apeleze un instrument. Utilizatorul trimite un mesaj în panoul Agent; frontend-ul face **POST /api/agent/chat/stream** și consumă Server Side Events (SSE). La evenimentul **tool_call**, backend-ul primește cererea de execuție de la containerul Docker prin **POST /api/tools/execute** și blochează pe **threading.Event**. Frontend-ul afișează modalul de aprobare; utilizatorul confirmă sau respinge. La confirmare, **execute()** rulează și rezultatul este trimis înapoi containerului, care îl introduce în contextul agentului și acesta continuă raționamentul.

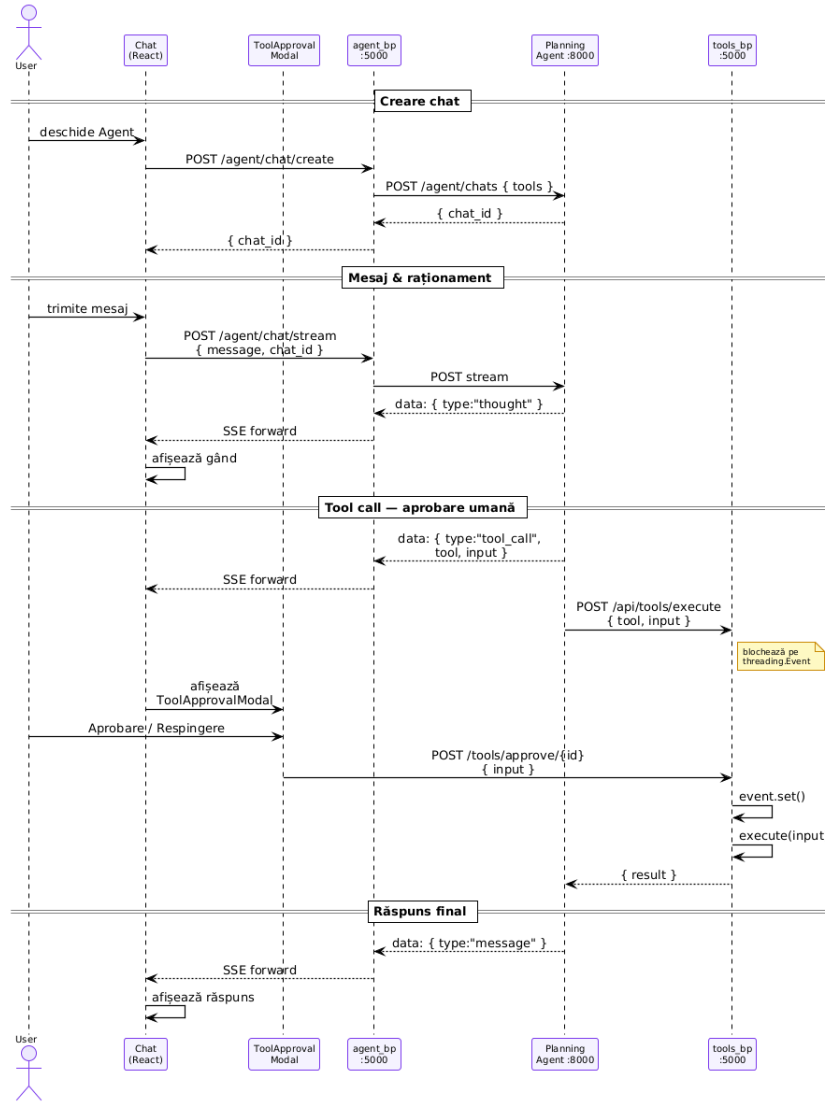


Figura 5.5: Secvența de execuție a unui instrument prin agent (human-in-the-loop)

5.3 Implementare

5.3.1 Tehnologii utilizate

Aplicația FileO combină un strat de prezentare web rulat în Electron cu un backend Python local și o serie de biblioteci specializate per categorie de fișier.

Stack-ul tehnologic al aplicației FileO este structurat pe trei componente principale: shell-ul desktop, backend-ul de procesare și instrumentele de care procesează și manipulează fișiere. Aplicația desktop este construită pe **Electron**, care gestionează procesul principal, fereastra nativă, comunicarea IPC și pornirea backend-ului Python, în timp ce interfața grafică este realizată în **React**, cu **TypeScript** pentru tipizare statică pe întregul cod frontend, **Vite** ca bundler rapid pentru dezvoltare și producție, iar navigarea client-side este gestionată cu **react-router-dom** prin **HashRouter**, fără a

necesita un server HTTP dedicat pentru rutare.

Backend-ul local este expus printr-un server REST **Flask**, organizat pe șabloane, cu suport pentru CORS și SSE. Operațiile de procesare a fișierelor sunt implementate prin biblioteci Python specializate: **Pillow** pentru conversia imaginilor raster (JPEG, PNG, WebP, BMP, TIFF, GIF), **rembg** pentru eliminarea fundalului prin modelul U2Net rulat local, **vtracer** pentru vectorizarea raster→SVG prin trasare de contur, **FFmpeg** (apelat prin subprocess) pentru conversia și compresia audio/video, **pypdf** pentru fuzionarea, divizarea, reordonarea și extragerea paginilor din fișiere PDF, **python-docx** pentru citirea și scrierea documentelor Word, și **Assimp/Open3D** pentru conversia modelelor 3D între formate (OBJ, GLB, FBX, STL etc.).

Pentru analiza rapidă a spațiului pe disc, aplicația accesează direct *Master File Table* (MFT) a sistemului NTFS prin **pywin32/ctypes**. Backend-ul Python este compilat într-un executabil standalone cu **PyInstaller**, iar instrumentele sunt acoperite de suite de teste unitare și de integrare scrise cu **pytest/unittest**.

Alegerea Electron ca shell de desktop a permis reutilizarea integrală a toolchain-ului web (React + Vite + TypeScript) fără a rescrie logica UI pentru un framework nativ. Backend-ul Flask a fost preferat față de alternativele asincrone (FastAPI) deoarece instrumentele de procesare de fișiere sunt în mare parte legate de I/O sincron și de subprocesses externe, iar simplitatea Flask reduce suprafața de configurare. Bibliotecile de procesare (Pillow, rembg, FFmpeg, pypdf) au fost alese pe criterii de maturitate, licență permisivă și absența dependenței de servicii externe, iar toate procesările au loc exclusiv local, pe mașina utilizatorului.

5.3.2 Pornirea aplicației și ciclul de viață

La lansarea aplicației, procesul Electron apelează `startPythonBackend()`, care detectează modul de rulare și pornește fie interpretorul Python cu `src/main.py`, fie executabilul compilat cu PyInstaller. Fereastra principală este creată imediat după și rămâne ascunsă până la evenimentul `ready-to-show`, eliminând efectul vizual de pâlپâit provenit din pornire. La închidere, `stopPythonBackend()` termină procesul fiu explicit, prevenind procese zombie. Monitorizarea drive-urilor externe rulează în paralel printr-un interval de 2 secunde care notifică `renderer`-ul prin IPC la orice modificare a literelor de disc disponibile.

5.3.3 Executorul de instrumente și aprobarea umană

Fiecare instrument (de imagine precum eliminare fundal sau convertire, audio de subtitrări, video de compresie, sau cele de vizualizare de spațiu digital) expune o funcție `execute(input: dict) -> str` care returnează un JSON serializat ce conține un câmp `success` indicând reușita operației, calea sau lista de căi către fișierele rezultate,

un câmp **error** cu mesajul de eroare în cazul unui eșec, precum și informații sau metrice specifice instrumentului respectiv (de exemplu numărul de fișiere procesate, formatul de ieșire sau scoruri de calitate ale rezultatului). Șablonul **tools_bp** menține un dicționar **_TOOLS** care mapează numele textual al instrumentului la modulul Python corespunzător și un set **_APPROVAL_REQUIRED** cu instrumentele care necesită confirmare umană înainte de execuție.

Când agentul solicită un instrument din **_APPROVAL_REQUIRED**, endpoint-ul **POST /api/tools/execute** creează un **threading.Event** și blochează thread-ul curent până la apelul **/approve/<id>** sau **/reject/<id>** din frontend, cu un timeout de 110 secunde. Utilizatorul vede cererea în interfață, poate modifica parametrii și confirmă sau respinge execuția. Dacă timeout-ul expiră fără răspuns, instrumentul este automat respins și agentul primește un mesaj de rejecție. Această arhitectură implementează un ciclu *human-in-the-loop* fără a bloca thread-ul principal Flask, fiecare cerere de execuție rulând pe un thread separat gestionat de serverul WSGI (Web Server Gateway Interface).

5.3.4 Procesarea fișierelor și modurile de output

Toate instrumentele de procesare a fișierelor suportă trei moduri de scriere a rezultatelor, codificate în câmpul **outputMode**: **replace** (înlocuiește fișierul original), **copy** (salvează rezultatul lângă original cu un sufix) și **virtual_drive** (plasează rezultatul într-un folder ierarhic gestionat de aplicație). Instrumentele cu volum mare de fișiere, conversii imagine, audio, video, documente, expun și o funcție **execute_parallel()** care distribuie fișierele pe un **ThreadPoolExecutor**, reducând timpul total de procesare pe sisteme multi-core.

Instrumentele AI (Image Enhancer, Audio Transcriber) nu execută inferența local, ci trimit datele către AI Gateway prin HTTP și retransmit răspunsul către frontend. Pentru a afișa progresul în timp real, aceste instrumente oferă și un endpoint **/stream** care implementează SSE : backend-ul consumă fluxul de la AI Gateway și îl redirecționează byte-cu-byte, fără buffering intermediar.

Instrumentele care procesează documente

Modulul **document_converter** asigură conversia bidirecțională între formate uzuale de document: PDF, DOCX, HTML, text simplu și Markdown. Nucleul său tehnic se sprijină pe un ansamblu de biblioteci specializate cu roluri bine delimitate. Extragerea și reconstrucția conținutului din fișiere PDF este realizată prin **pypdf** [FSpz⁺24], care expune un model orientat pe obiecte al structurii interne PDF, permițând accesul paginat la conținut. Conversia dintre formatul PDF și DOCX se realizează în ambele direcții: **pdf2docx** reconstituie documentul Word păstrând, pe cât posibil, aspectul vizual original, în timp ce **docx2pdf** delegă redarea finală motorului de tipărire al aplicației Microsoft Word, garantând fidelitatea tipografică. Redarea vizuală a paginilor

PDF sub formă de imagini raster este asigurată de PyMuPDF (`fitz`), o legătură Python peste motorul MuPDF, capabilă să producă reprezentări la rezoluție configurabilă. Manipularea documentelor Word la nivel structural, paragrafe, stiluri, tabele, este gestionată de `python-docx`, în timp ce conversia inversă din HTML spre DOCX folosește biblioteca `mammoth`, care interpretează marcajul semantic al documentelor Word. Generarea de fișiere PDF pornind de la conținut textual simplu este delegată lui `reportlab`, o bibliotecă de compunere tipografică ce permite controlul precis al aspectului paginii, iar randarea HTML în PDF este asigurată de `WeasyPrint`, care implementează un subset relevant din specificațiile CSS pentru medii de tipărire. Procesarea fișierelor Markdown beneficiază de biblioteca `markdown`, ce suportă extensii pentru tabele, blocuri de cod și generare de cuprins, folosită ca etapă intermediară în lanțul Markdown → HTML → PDF sau DOCX.

Modulul `pdf_merger` extinde capacitățile de manipulare a documentelor dincolo de simpla conversie de format, adresând operații structurale asupra fișierelor PDF: îmbinarea mai multor documente într-unul singur, cu adăugarea automată de marcate de navigație (*bookmarks*), extragerea selectivă de pagini pe baza unor intervale specificate și reordonarea paginilor. Toate aceste operații sunt implementate prin intermediul aceleiași biblioteci `pypdf`, care asigură un nivel de abstractizare suficient pentru a manipula structura internă a documentului fără a degrada conținutul. Arhitectura ambelor module urmează un model comun de execuție: un planificator de sarcini bazat pe `ThreadPoolExecutor` permite procesarea paralelă a loturilor de fișiere, izolând totodată erorile individuale pentru a nu afecta restul operațiilor. Rezultatele sunt serializate în format JSON și includ metadate despre succesul fiecărei operații, căile de ieșire generate și eventualele mesaje de eroare, facilitând integrarea modulelor în fluxuri de procesare automatizată.

Instrumentul ce mărește rezoluția imaginilor cu AI

Îmbunătățirea calității vizuale a imaginilor digitale este realizată prin integrarea modelului **Swin2SR** [CCBT22] (*Swin V2-based Intermediate-layer Restorer*), o arhitectură de ultimă generație ce utilizează mecanisme de atenție ierarhică pentru sarcini de restaurare a imaginilor. Implementarea specifică, bazată pe modelul `swin2SR-classical-sr-x2-64`¹, vizează super-rezoluția clasică prin aplicarea unui factor de scalare $x2$ asupra intrărilor cu rezoluție scăzută. Spre deosebire de modelele convoluționale tradiționale, această arhitectură utilizează blocuri de tip *Swin Transformer* cu ferestre glisante (*shifted windows*), permițând capturarea dependențelor spațiale pe distanțe lungi și conservarea texturilor fine prin procesarea reziduurilor de înaltă frecvență. Procesul tehnic implică o etapă de *patch embedding*, urmată de o serie de straturi de procesare neliniară a

¹<https://huggingface.co/caidas/swin2SR-classical-sr-x2-64>

caracteristicilor și o reconstrucție finală bazată pe straturi de *sub-pixel convolution*, garantând astfel o fidelitate superioară a detaliilor reconstruite și o atenuare eficientă a artefactelor de compresie.

În cadrul componentei de procesare a limbajului natural, a fost integrat modelul **Whisper** [RKX⁺22] pentru automatizarea procesului de transcriere a fișierelor audio încărcate în aplicație. Alegerea acestui model a fost determinată de capacitatea sa ridicată de generalizare și de performanța constantă în conversia datelor audio în format text, indiferent de variațiile de accent sau zgomot de fundal prezente în secvențele de intrare. Rolul implementării a constat în dezvoltarea unei conduite (*pipeline*) de date care preia eșantioanele audio, le normalizează și interfățează cu modelul pentru a returna un transcript precis în multiple limbi precum engleză, română, spaniolă etc. (există dectarea automată a limbii).

Modelul utilizat se bazează pe o arhitectură de tip *Transformer* [VSP⁺17] cap la coadă (*end-to-end*), antrenată pe un set de date masiv de 680.000 de ore de înregistrări multilingve și sarcini multiple [RKX⁺22]. Fundamentul metodologic propus de autori demonstrează că utilizarea unei supravegheri slabe la scară largă (*large-scale weak supervision*) permite modelului să depășească performanțele sistemelor antrenate pe seturi de date mai mici, dar etichetate riguros. Arhitectura implementează un encoder care procesează spectrograma log-Mel a semnalului audio și un decoder care generează secvențial token-urile de text, utilizând mecanisme de atenție pentru a prezice corect transcrierea, identificarea limbii și a marcajelor temporare aferente.

5.3.5 Drive-ul virtual

Drive-ul virtual este un folder ierarhic, gestionat de modulul `utils/drive_manager.py` și înregistrat în `data/known_drives.json`. La crearea unui drive prin instrumentul *Drive Creator*, aplicația scanează folder-ul sursă folosind MFT (*Master File Table*), implementat în `utils/mft_scan.py` ca apel direct la API-ul Win32 pentru citire NTFS la nivel raw. Această abordare este semnificativ mai rapidă decât traversarea recursivă a directoarelor prin `os.walk()` pentru foldere cu zeci de mii de fișiere. Fișierele identificate sunt adăugate în drive fie ca scurtături `.lnk` (acțiunea `shortcuts`), fie prin mutare fizică (acțiunea `move`).

Instrumentul de creare a drive-urilor virtuale implementează o strategie hibridă de gestionare a datelor, operând în două moduri distincte la nivelul sistemului de fișiere NTFS: modul *move*, care realizează relocalizarea fizică a resurselor prin intermediul funcției `shutil.move()`, și modul *shortcut*, bazat pe generarea de fișiere de tip Windows Shell Link (`.lnk`). Pentru modul *shortcut*, procesul de creație recurge la execuția unor script-uri VBScript generate dinamic, invocate prin interpretorul `cscript.exe` cu parametrul `//NoLogo`. Această abordare utilizează obiectul `COM WScript.Shell` și

metoda `CreateShortcut()`, asigurând conformitatea cu specificațiile API-ului Windows Shell.

Eficiența acestui sistem este susținută de un tool de scanare a volumelor care exploatează direct Master File Table (MFT) prin apeluri de nivel scăzut la API-urile Win32, evitând limitările traversării recursive convenționale. Accesul la MFT se realizează prin deschiderea unui *handle* brut asupra volumului fizic (`GENERIC_READ`) utilizând `CreateFileW()`, și `FSCTL_GET_NTFS_VOLUME_DATA` (`0x00090064`) a urmat să fie emis prin `DeviceIoControl()`. Înregistrările sunt citite secvențial în blocuri de 1 MB și parsate binar pentru extragerea atributelor `$STANDARD_INFORMATION` (`0x10`), `$FILE_NAME` (`0x30`) și `$DATA` (`0x80`). Reconstituirea căilor se realizează iterativ printr-un dicționar indexat pe numărul de *record*, eliminând riscul de *stack overflow*.

5.3.6 Comunicarea dintre agent și aplicație

Când utilizatorul trimite un mesaj, frontend-ul apelează `POST /api/agent/chat/stream`. Backend-ul deschide un flux SSE către serverul de planificare, injectând în fiecare definiție de instrument `callback_url = "http://host.docker.internal:5000/api/tools/execute"`. Această adresă permite containerului Docker al serverului să apeleze executorul local al APP-ului fără ca portul 5000 să fie expus în rețeaua externă. Agentul emite evenimente de tip *llm_chunk* (text generat de model, inclusiv raționamentul), *tool_call* (apel instrument cu argumente), *tool_result* (răspuns instrument) și *final* (răspuns final). Frontend-ul randează fiecare tip de eveniment diferit: textul generat în pași de tip "llm" este afișat colapsabil, apelurile de instrumente care necesită aprobare declanșează modalul `ToolApprovalModal`, iar răspunsul final este afișat în câmpul de chat.

5.4 Testare

Strategia de testare a aplicației FileO acoperă două niveluri complementare: testarea automată a instrumentelor backend prin suite PyUnit și testarea componentelor AI prin evaluări cantitative descrise. Cele două niveluri sunt deliberat separate: instrumentele deterministe pot fi verificate exhaustiv cu date sintetice, în timp ce modelele AI necesită seturi de date reale și metrice specializate (Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), Word Error Rate (WER)) care nu se pretează testelor unitare clasice.

5.4.1 Performanța componentelor AI

Instrumentele *Image Enhancer* și *Audio Transcriber* din FileO sunt în esență interfețe grafice peste modelele Swin2SR și Whisper Large V3 rulate pe AI Gateway. Swin2SR a fost evaluat pe un set de 855 de perechi LR/HR cu metrice full-reference (PSNR,

SSIM, MAE) și metrici perceptuale (Delta E, sharpness, edge density), iar Whisper a fost caracterizat structural prin numărul de parametri, viteza de inferență (RTF) și suportul multilingv. Rezultatele respective sunt valabile direct pentru componentele AI ale aplicației FileO, deoarece apelul din APP nu modifică parametrii modelului, ci doar proxiază datele.

5.4.2 Suite PyUnit pentru instrumente

Instrumentele deterministe ale aplicației sunt acoperite de 12 suite `unittest.TestCase`, câte una per modul de tool, plasate în `APP/tests/`. Suitele sunt structurate uniform în trei clase per fișier, urmând un pattern consistent aplicat tuturor instrumentelor.

Structura unui fișier de test

Fiecare fișier de test urmează același tipar cu trei clase:

- **TestXxxDefinition**, verifică schema `DEFINITION` expusă agentului: prezența cheilor obligatorii (`name`, `description`, `parameters`), corectitudinea enumerărilor (`outputMode: replace | copy | virtual_drive`), și consistența câmpurilor `required`.
- **TestXxxInputValidation**, verifică comportamentul la intrări invalide fără a depinde de biblioteci externe sau fișiere pe disc: lista goală de fișiere, chei lipsă, căi inexistente, formate nesuportate, `outputPath` gol în modul `virtual_drive`. Toate aceste cazuri trebuie să returneze `success: false` cu un câmp `error` prezent, fără excepții negestionate.
- **TestXxxIntegration**, teste de integrare reale, decorate cu `@unittest.skipUnless` pentru fiecare dependență opțională (Pillow, pypdf, FFmpeg, Assimp etc.) și pentru prezența fișierelor de test din `APP/test/`. Fiecare test rulează în `tempfile.mkdtemp()` cu cleanup în `tearDown()`, garantând izolarea completă între teste.

Cazuri de test reprezentative

Ca ilustrare a profunzimii testării, suite-le acoperă scenarii precum conversii între toate formatele suportate, cu verificarea extensiei și existenței fișierului de output; efectul parametrului `quality` asupra dimensiunii fișierului rezultat (calitate 95 vs. 10 produce fișiere JPEG de dimensiuni semnificativ diferite); procesarea în lot (*batch*), cu contorizare corectă a câmpurilor `total`, `succeeded` și `failed`, inclusiv în scenariul de eșec parțial (un fișier valid și un fișier inexistent); modul `virtual_drive`, în care folderul de drive este creat, fișierul de output este plasat în interiorul acestuia, iar calea este returnată în câmpul `virtualDrivePath`; precum și verificarea faptului că funcția `execute_parallel()` returnează aceeași structură JSON ca `execute()`.

Rularea suitei complete

Suite-le se execută cu pytest din rădăcina proiectului: `python -m pytest APP/tests/-v`. Testele de validare a intrărilor și a schemei **DEFINITION** rulează fără nicio dependență externă și sunt adecvate pentru un mediu integrare continuă. Testele de integrare sunt sărite automat (**skipUnless**) dacă biblioteca necesară sau fișierul de date lipsesc, fără a eșua suitele.

Suitele de teste PyUnit ale instrumentelor FileO acoperă cinci categorii. Pentru imagine (conversie, vectorizare, eliminare fundal), dependențele de integrare sunt Pillow, rembg și vtracer, iar scenariile testate vizează formatele suportate, procesarea pe loturi (*batch*), parametrul *quality* și modul *virtual drive*. Pentru audio/video, testele depind de FFmpeg (apelat prin subprocess) și acoperă conversia între formate, compresia și contorizarea eșecurilor. Pentru documente (PDF, DOCX), dependențele sunt pypdf și python-docx, cu scenarii de fuzionare, divizare și reordonare a paginilor. Pentru conversia modelelor 3D, testele depind de Assimp/Open3D și verifică formatele OBJ, GLB și STL, precum și eșecul controlat la un format invalid. În final, pentru instrumentele de sistem (creare de unități virtuale, analiză a spațiului pe disc), dependența de integrare este API-ul Win32/NTFS pe Windows, iar scenariile acoperă crearea unității virtuale și diferența dintre modul *shortcuts* și modul *move*.

Capitolul 6

Server pentru Agent și Autentificarea Clienților. Arhitectură și Implementare

6.1 Specificarea Aplicației

Serverul reprezintă un backend centralizat care asigură autentificarea utilizatorilor, procesarea conținutului prin modele AI și executarea autonomă a task-urilor printr-un agent cu planificare. Sistemul interacționează cu doi actori: *Utilizatorul* (persoana fizică ce accesează serverul prin interfața web sau prin aplicația FileO) și *Aplicația client (FileO)* (componenta software care comunică automatizat, autentificată prin cheie API).

6.1.1 Cerințe Funcționale

Cerințele sunt grupate în patru arii: autentificare, gateway AI, agent inteligent și interfață web.

Autentificare și gestionare cont

- CF-01** Sistemul trebuie să permită înregistrarea unui utilizator nou pe baza unei adrese de email valide și a unei parole cu lungimea minimă de 8 caractere, returnând un token de acces JWT.
- CF-02** Sistemul trebuie să permită autentificarea unui utilizator înregistrat prin verificarea credențialelor și emiterea unui token JWT cu durată de valabilitate configurabilă.
- CF-03** Sistemul trebuie să ofere utilizatorului autentificat posibilitatea de a vizualiza datele propriului cont.

CF-04 Sistemul trebuie să permită utilizatorului autentificat să genereze o cheie API unică, destinată comunicării machine-to-machine cu serviciul de agent.

CF-05 Sistemul trebuie să permită revocarea cheii API și ștergerea sesiunii de agent asociate.

Gateway AI

CF-06 Sistemul trebuie să permită scalarea imaginilor la rezoluție dublă ($\times 2$) utilizând modelul Swin2SR, returnând imaginea procesată codificată în Base64 împreună cu metrici de calitate (PSNR, SSIM, sharpness).

CF-07 Sistemul trebuie să permită transcrierea fișierelor audio în text utilizând modelul Whisper Large V3, cu suport pentru detecția automată a limbii și calculul metricilor WER/CER când este furnizat textul de referință.

CF-08 Sistemul trebuie să permită generarea de text pe baza unui prompt utilizând modelul de limbaj Groq (`llama-3.3-70b-versatile`), cu parametri configurabili de temperatură și număr maxim de tokeni, expus atât în variantă sincronă (`/llm/generate`) cât și cu streaming SSE token-cu-token (`/llm/stream`).

CF-08b Sistemul trebuie să permită analiza vizuală a imaginilor utilizând modelul Groq Llama 4 Scout 17B (`meta-llama/llama-4-scout-17b-16e-instruct`), returnând o descriere detaliată a conținutului și o listă de cuvinte-cheie extrase automat.

CF-09 Sistemul trebuie să permită generarea de subtitrări în formatele SRT și VTT dintr-un fișier video, prin extragerea pistei audio cu FFmpeg și transcrierea acesteia cu Whisper.

CF-10 Sistemul trebuie să expună endpoint-uri pentru pre-încărcarea și descărcarea din memorie a fiecărui model AI, permițând gestionarea resurselor RAM/VRAM.

Sistemul de agent inteligent

CF-11 Sistemul trebuie să permită aplicației client să trimită mesaje unui agent conversațional, însoțite de o listă de tool-uri disponibile și URL-urile de callback HTTP corespunzătoare.

CF-12 Sistemul trebuie să permită agentului să execute tool-uri din lista primită prin apelarea URL-ului de callback asociat, transmițând parametrii de intrare și returnând rezultatul aplicației client.

- CF-13** Sistemul trebuie să permită agentului să planifice și să execute task-uri complexe în mai mulți pași, utilizând un model Plan-and-Execute cu noduri de planificare, execuție și replanificare adaptivă.
- CF-14** Sistemul trebuie să pună la dispoziția agentului două tool-uri built-in predefinite: editarea fișierelor text și mutarea/redenumirea fișierelor în drive-ul virtual FileO, executate prin callback la aplicația client.
- CF-15** Sistemul trebuie să permită resetarea istoricului conversației pentru o sesiune de agent activă, fără ștergerea cheii API asociate.

Distribuție container și interfață web

- CF-16** Sistemul trebuie să expună o interfață web de administrare pentru autentificare, descărcarea aplicației și accesul la documentație.

Cerințele funcționale sunt ilustrate prin trei diagrame Use Case UML, fiecare cu *system boundary*, actori și relații «include»/«extend».

Diagrama UC-1: Autentificare și gestionare cont

Prima diagramă (Figura 6.1) surprinde interacțiunile cu modulul de autentificare. Generarea și revocarea cheii API includ autentificarea prealabilă («include»).

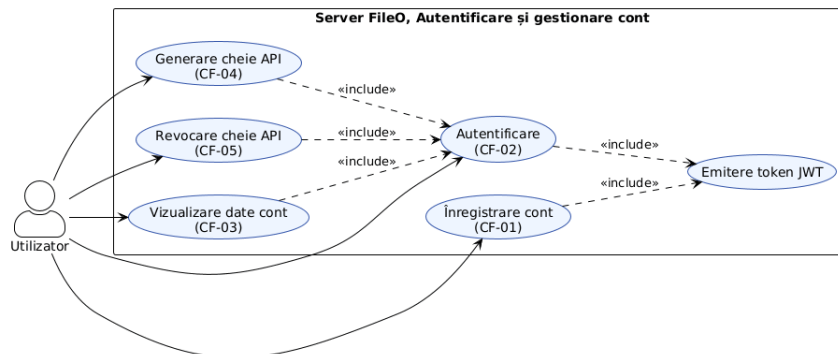


Figura 6.1: Diagrama Use Case UC-1: Autentificare și gestionare cont

Diagrama UC-2: Gateway AI

A doua diagramă (Figura 6.2) acoperă funcționalitățile AI expuse ambilor actori. Generarea subtitrărilor include implicit transcrierea prin Whisper («include»).

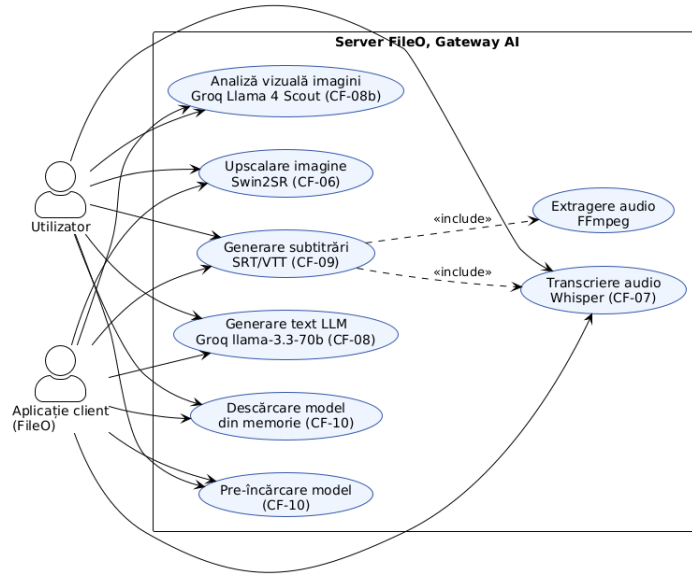


Figura 6.2: Diagrama Use Case UC-2: Gateway AI

Diagrama UC-3: Sistemul de agent inteligent

A treia diagramă (Figura 6.3) modelează interacțiunile cu sistemul de agent. Activarea modului de planificare extinde cazul de utilizare de bază («**extend**»); replanificarea derivă din crearea planului («**include**»).

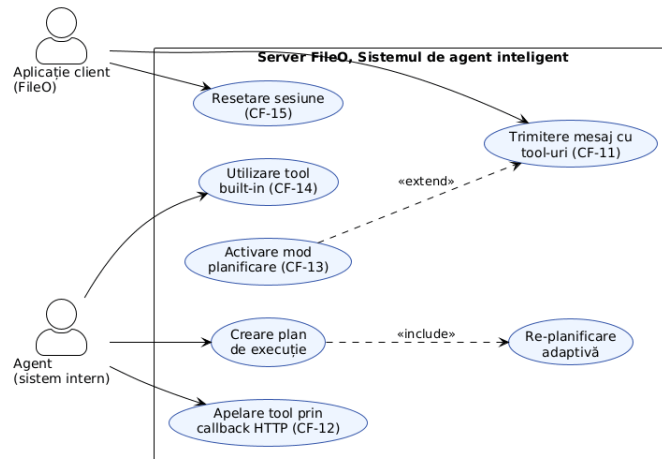


Figura 6.3: Diagrama Use Case UC-3: Sistemul de agent inteligent

6.1.2 Cerințe Non-Funcționale

Cerințele nefuncționale ale serverului acoperă șase dimensiuni principale. Din punct de vedere al performanței, autentificarea trebuie finalizată în cel mult 200,ms, cererile către agent în cel mult 120,s, iar modelele AI sunt încărcate lazy, la primul apel. La nivel de scalabilitate, serverul FastAPI gestionează cererile concurente prin event loop-ul

`asyncio`, delegând operațiile sincrone de I/O și inferență prin `asyncio.to_thread()` pentru a nu bloca thread-ul principal. Pentru securitate, parolele sunt stocate ca hash bcrypt, autentificarea se bazează pe tokenuri JWT semnate HMAC-SHA256, iar fiecare utilizator primește o cheie API unică (**X-API-Key**). În privința disponibilității, PostgreSQL rulează local într-un container Docker cu volum persistent, iar în producție containerele repornesc automat. Pentru mentenanță, modelele AI respectă interfața `BaseAIModel` (principiul OCP — *Open/Closed Principle*, "deschis pentru extensie, închis pentru modificare"), schema bazei de date este versionată prin migrații `Alembic`, iar configurarea se face prin fișiere `.env`. Portabilitatea este asigurată prin compatibilitatea cu Python 3.11+ rulat nativ sau prin Docker Compose.

6.2 Proiectarea sistemului

6.2.1 Arhitectura generală

Serverul este structurat pe trei straturi interne, grupate sub un pachet *Server*: *API Layer* (FastAPI + Uvicorn, SPA React), *Business Logic Layer* (Auth, AI Gateway, Planning Agent, Chat Manager) și *Infrastructure Layer* (SQLAlchemy ORM + Alembic). Serviciile externe: *PostgreSQL* (rulat local într-un container Docker), *Groq API* (llama-3.3-70b-versatile și Llama 4 Scout pentru viziune) și *HuggingFace Hub* (Swin2SR, Whisper) sunt accesate exclusiv din stratul de Business Logic. Interfața web React/Vite este servită static direct de FastAPI. Arhitectura componentelor este ilustrată în Figura 6.4.

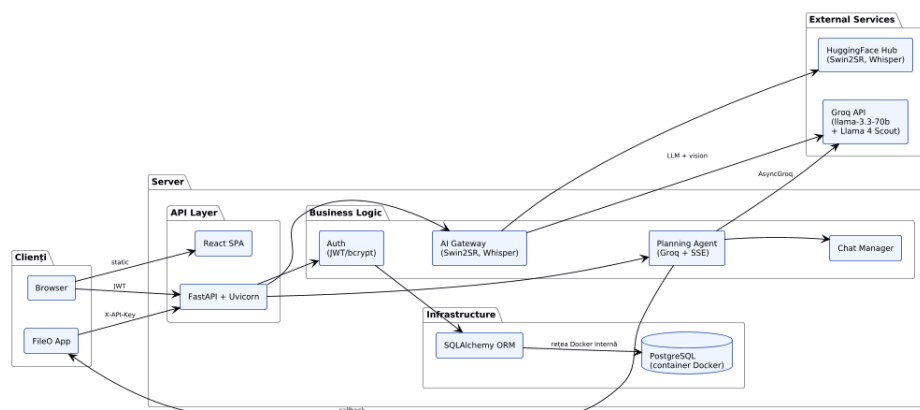


Figura 6.4: Diagrama de componente a serverului

Infrastructura de rulare

Configurația curentă plasează serverul FastAPI pe un docker local (`uvicorn`, portul 8000), cu PostgreSQL rulând într-un container Docker izolat, datele fiind persistate

într-un volum Docker dedicat (Figura 6.5).

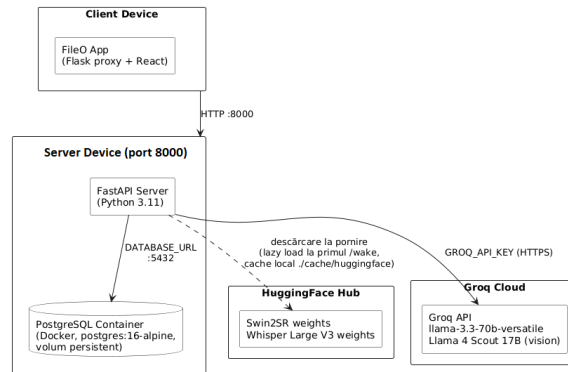


Figura 6.5: Diagrama de deployment: infrastructura curentă

6.2.2 Diagrame de clase

Modelul de date

Persistența este asigurată prin SQLAlchemy ORM cu două entități: **User** și **AgentKey**, legate printr-o relație one-to-one cu politică **CASCADE**. `hashed_password` stochează exclusiv hash-ul bcrypt; `api_key` este un UUID indexat pentru căutare rapidă (Figura 6.6).

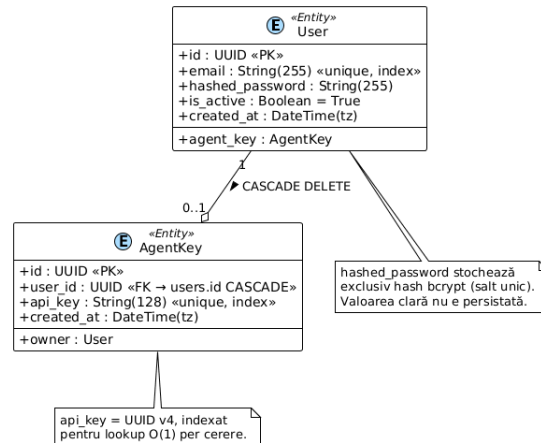


Figura 6.6: Diagrama de clase: modelul de date ORM

Ierarhia modelelor AI

Clasa abstractă **BaseAIModel** definește contractul comun (`wake_up()`, `process()`, `unload()`) și lazy loading thread-safe prin `asyncio.Lock`. Ierarhia curentă cuprinde **Swin2SRModel** și **WhisperModel** (Figura 6.7). Generarea de text și analiza vizuală prin Groq nu fac parte din această ierarhie, sunt apelate direct în router prin **AsyncGroq**, fără lazy loading local.

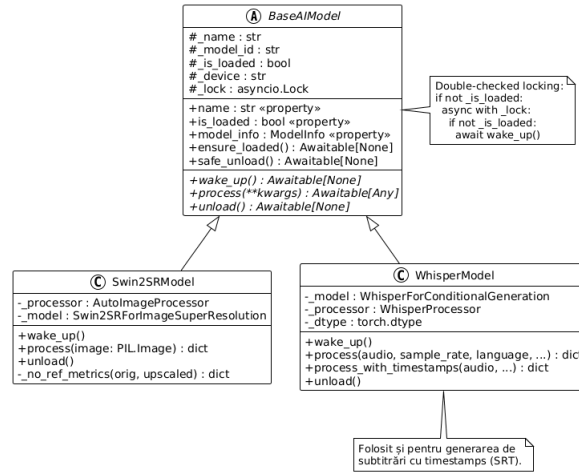


Figura 6.7: Diagrama de clase: ierarhia modelelor AI

Sistemul de agent

Sistemul de agent este structurat în jurul clasei **PlanningAgent** (planifică, execută și sintetizează răspunsul prin API-ul Groq) și al clasei **ChatManager** (gestionează istoricul conversațiilor, organizat ca **Chat** cu liste de **Message**). Planul generat este reprezentat printr-o listă de obiecte **Step**, iar apelurile de tool-uri folosesc schema **ToolDefinition** (schemă Pydantic cu **name**, **description**, **parameters** și **callback_url**) (Figura 6.8). Tool-urile built-in **TextEditorTool** și **VirtualDriveMoveTool** implementează această interfață.

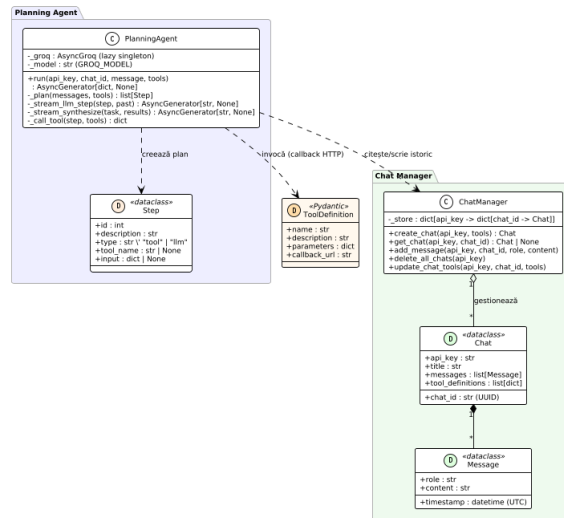


Figura 6.8: Diagramă de clase: Sistemul de agent (PlanningAgent)

6.2.3 Diagrame de secvență

Fluxul de autentificare JWT

La înregistrare (Figura 6.9), serverul validează unicitatea email-ului, calculează hash-ul bcrypt și returnează un JWT semnat. La autentificare, verifică hash-ul cu `bcrypt.checkpw()` și emite un nou token. Endpoint-urile protejate verifică semnătura fără interogarea bazei de date.

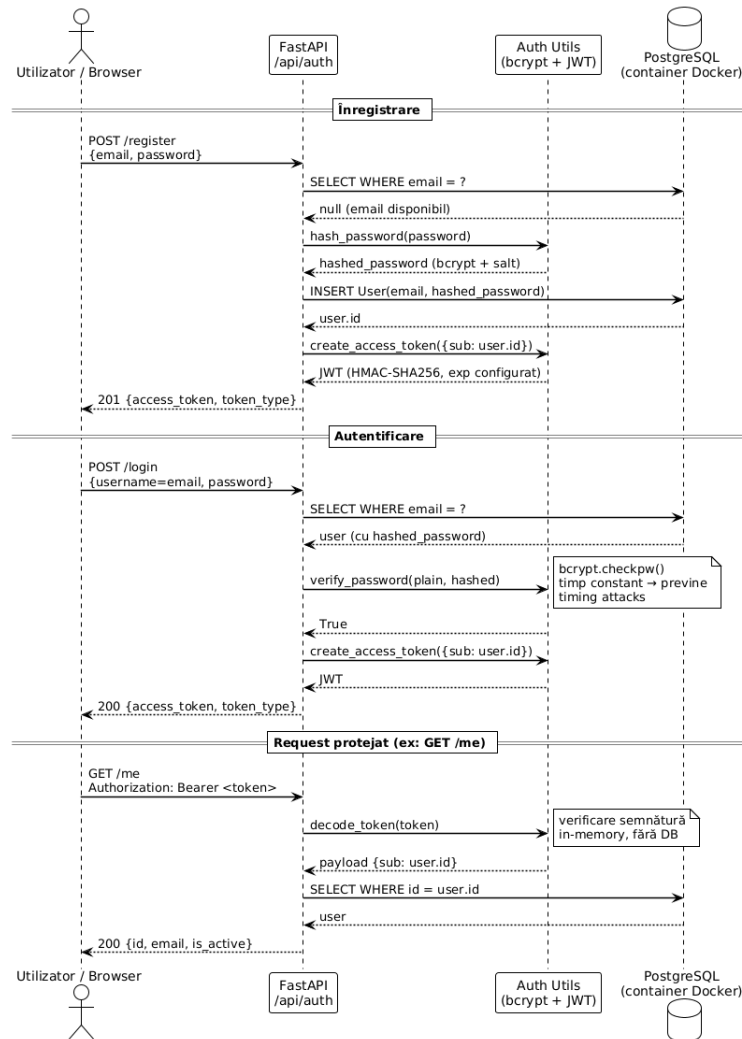


Figura 6.9: Diagrama de secvență: autentificare JWT

Fluxul agentului cu planificare (Plan-and-Execute)

Agentul de planificare (Figura 6.10) procesează fiecare cerere în trei faze secvențiale prin API-ul Groq cu streaming SSE: faza de *planificare* în care modelul generează un plan JSON cu pași de tip `tool` sau `llm`, faza de *execuție* în care fiecare pas este procesat în ordine, apelând callback-ul tool-ului sau generând text, și faza de *sinteză* în care Groq formulează răspunsul final din rezultatele tuturor pașilor. Fiecare eveniment al execuției este transmis clientului în timp real prin Server-Sent Events.

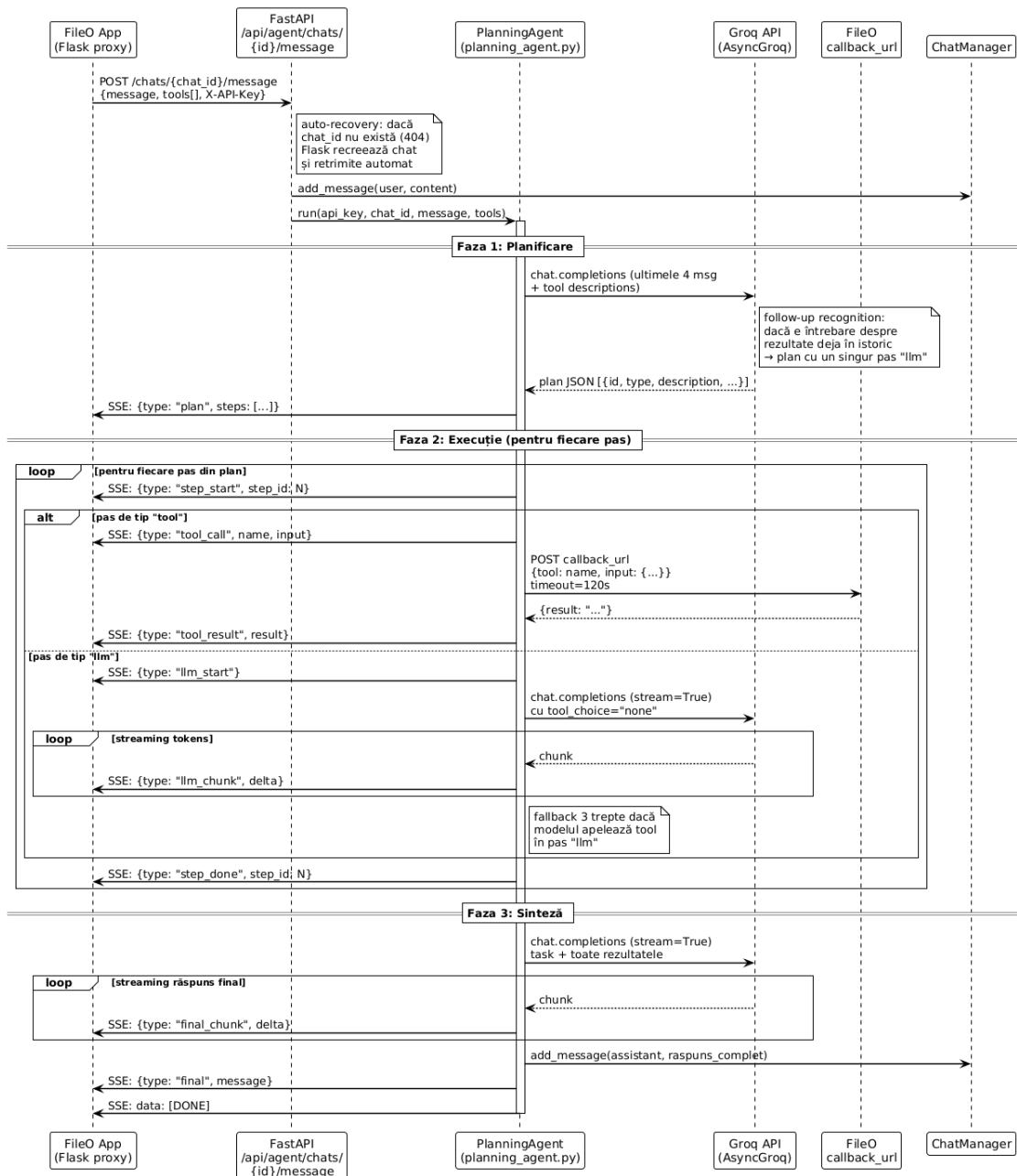


Figura 6.10: Diagrama de secvență: agentul cu planificare Plan-and-Execute

6.3 Implementare

6.3.1 Tehnologii utilizate

Stack-ul tehnologic al serverului este structurat pe trei componente principale: backend, agent/AI și frontend de administrare. Pe partea de backend, **FastAPI** oferă framework-ul web REST asincron, cu rutare, validare automată a datelor și documentație Swagger UI generată automat, rulat printr-un server ASGI **Uvicorn**. Accesul la baza de date se face prin ORM-ul **SQLAlchemy**, cu versionarea și migrarea schemei gestionate de

Alembic; baza de date propriu-zisă este **PostgreSQL**, rulată local într-un container Docker cu volum persistent. Validarea datelor de intrare/ieșire și gestionarea setărilor sunt realizate cu **Pydantic**, autentificarea se bazează pe tokenuri JWT generate și verificate cu **python-jose**, iar parolele sunt hash-uite securizat cu **Passlib/bcrypt**.

Pe partea de agent și inteligență artificială, agentul de planificare, gateway-ul LLM și modulul de analiză vizuală folosesc clientul Python asincron oficial **groq-python**. Modelele de super-rezoluție (Swin2SR) și de transcriere audio (Whisper) sunt încărcate și rulate prin **HuggingFace Transformers**, cu **PyTorch** ca backend de inferență. Apelurile callback către tool-urile aplicației client folosesc clientul HTTP asincron **httpx**. Pentru extragerea pistei audio din fișierele video este utilizat **FFmpeg**.

Interfața web de administrare este o aplicație **React**, scrisă în **TypeScript**, construită și servită în dezvoltare cu **Vite** și stilizată utility-first cu **Tailwind CSS**. La nivel de infrastructură, **Docker** este folosit pentru containerizarea PostgreSQL și pentru packaging-ul aplicației client, iar **Docker Compose** orchestrează serviciile în mediul de dezvoltare.

Serverul rulează pe docker local (**uvicorn**, portul 8000). PostgreSQL rulează local într-un container Docker (image **postgres:16-alpine**, identic cu serviciul **postgres** din **docker-compose.yml**), cu datele persistate într-un volum Docker dedicat. Greutățile Swin2SR și Whisper sunt descărcate la primul apel **/wake** și reutilizate din cache-ul local HuggingFace la accesările ulterioare.

6.3.2 Modelul de date și persistența

Persistența datelor este asigurată prin SQLAlchemy ORM, utilizând două entități principale: **User** și **AgentKey**. Entitatea **User** reține adresa de email (coloană unică, indexată), hash-ul bcrypt al parolei și starea contului (**is_active**). Entitatea **AgentKey** este legată de **User** printr-o relație bidirecțională one-to-one cu politică **CASCADE** la ștergere, garantând că revocarea contului elimină automat și cheia API asociată. Coloana **api_key** stochează un UUID v4 indexat, ce permite căutarea în timp constant la fiecare cerere autenticată prin **X-API-Key**.

Sesiunile de bază de date sunt gestionate prin dependența **get_db()**, implementată cu pattern-ul **yield**: sesiunea este furnizată handler-ului și închisă garantat în blocul **finally**, indiferent dacă cererea s-a finalizat cu succes sau cu eroare. Parametrii pool-ului de conexiuni, aceștia fiind: **pool_size=2** (numărul de conexiuni persistente menținute permanent în pool), **max_overflow=5** (numărul maxim de conexiuni suplimentare, temporare, create peste **pool_size** în perioadele de vârf și închise după utilizare) și **pool_recycle=300** (intervalul, în secunde, după care o conexiune este reciclată automat, pentru a evita conexiunile expirate de partea bazei de date), sunt calibrați pentru o utilizare moderată a conexiunilor, adecvată unei instanțe PostgreSQL

rulete local într-un container Docker cu resurse limitate.

Migrările de schemă sunt gestionate prin Alembic, permițând evoluția structurii bazei de date fără pierderea datelor existente și fără intervenție manuală în mediul de producție.

6.3.3 Sistemul de autentificare

Autentificarea se bazează pe două mecanisme complementare: JWT pentru sesiunile utilizatorilor din interfața web și chei API pentru comunicarea machine-to-machine dintre aplicația FileO și server.

La înregistrare, serverul verifică unicitatea adresei de email în baza de date și validează că parola respectă lungimea minimă de 8 caractere. Parola nu este niciodată stocată în clar, se persistează exclusiv hash-ul `bcrypt`, calculat cu un salt generat aleatoriu la fiecare înregistrare. În urma persistării cu succes, serverul emite direct un token JWT semnat cu HMAC-SHA256, eliminând un pas suplimentar de autentificare.

La autentificare, serverul compară parola cu hash-ul stocat prin `bcrypt.checkpw()`, o funcție de comparație în timp constant ce previne atacurile de tip timing. Tokenul JWT emis încorporează identificatorul utilizatorului (`sub`) și un timestamp de expirare (`exp`) configurabil prin variabile de mediu. Endpoint-urile protejate verifică semnătura tokenului exclusiv în memorie, fără a interoga baza de date la fiecare cerere, ceea ce menține latența de autentificare sub 200 ms.

Cheia API este un UUID v4 generat la cererea explicită a utilizatorului. Aceasta este transmisă în header-ul `X-API-Key` de către aplicația FileO și verificată prin interogare directă a tabelului `agent_keys`. Revocarea cheii este imediată și determină eșuarea tuturor cererilor în curs ale sesiunii asociate.

6.3.4 Gateway-ul de modele AI

Toate modelele AI sunt încărcate la cerere (*lazy loading*), nu la pornirea serverului, pentru a evita consumul inutil de memorie RAM și VRAM în perioadele de inactivitate. Mecanismul este implementat prin metoda `ensure_loaded()`, care aplică pattern-ul *double-checked locking*: o primă verificare rapidă a flag-ului `_is_loaded` evită achiziția inutilă a lock-ului în cazul tipic (modelul deja în memorie); o a doua verificare, efectuată după achiziția `asyncio.Lock`, previne apelul dublu al `wake_up()` în scenariile de concurență ridicate. Descărcarea din memorie urmează același protocol prin `safe_unload()`, garantând că un model nu este descărcat în timp ce un alt coroutine îl utilizează.

Modelul pentru scalarea imaginilor este `caidas/swin2SR-classical-sr-x2-64`, descărcat de pe HuggingFace Hub la primul apel al endpoint-ului `/wake` și stocat în cache-ul local pentru accesările ulterioare. Încărcarea propriu-zisă (instanțierea procesorului și

a modelului, transferul pe dispozitivul de inferență) rulează prin `asyncio.to_thread()`, astfel încât event loop-ul FastAPI nu este blocat în timpul operației sincrone de I/O și calcul. La inferență, modelul primește imaginea de intrare codificată în Base64, realizează upscaling-ul $\times 2$ și returnează imaginea rezultată tot în Base64.

Modelul Whisper Large V3, distribuit de OpenAI prin HuggingFace, este utilizat atât pentru transcrierea directă a fișierelor audio (endpoint-ul `/transcribe`), cât și pentru generarea subtitrărilor din fișiere video (endpoint-ul `/subtitles`). În al doilea caz, pista audio este extrasă în prealabil prin FFmpeg. Modelul suportă detectarea automată a limbii vorbite sau specificarea explicită a unui cod ISO-639. La fiecare inferență se calculează *Real-Time Factor* (RTF), raportul dintre durata procesării și durata clipului audio, ca indicator de performanță. Dacă este furnizat un text de referință, se calculează suplimentar *Word Error Rate* (WER) și *Character Error Rate* (CER) prin distanța Levenshtein, utile în scenariile de evaluare a calității transcrierii.

Generarea de text și analiza vizuală sunt expuse prin același client `AsyncGroq` utilizat de agentul de planificare, reutilizând cheia `GROQ_API_KEY` și modelul configurat prin

6.3.5 Sistemul de agent cu planificare

Arhitectura generală

Sistemul de agent al serverului implementează un pipeline Plan-and-Execute personalizat, alimentat de API-ul Groq și cu suport complet pentru streaming SSE, descris în detaliu în paragrafele următoare.

Chat Manager și modelul de conversație

Gestionarea conversațiilor este realizată de modulul `chat_manager`, care introduce conceptul de *chat* ca unitate centrală de interacțiune. Un `Chat` este o structură de date ce conține un identificator UUID, cheia API a proprietarului, un titlu generat automat din primele 60 de caractere ale primului mesaj al utilizatorului, lista cronologică a mesajelor (`Message`) din sesiune și lista definițiilor tool-urilor active pentru acea sesiune. Structura `Message` reține rolul (`user`, mesajul trimis de utilizator, `assistant`, răspunsul generat de agent, sau `system`, instrucțiuni de configurare transmise modelului), conținutul și un timestamp UTC.

Stocarea este exclusiv în memorie (*in-memory store*), organizată ca un dicționar dublu indexat pe cheia API și pe identificatorul chat-ului. Această decizie de design simplifică schema bazei de date, cu compromisul că istoricul conversațiilor se pierde la repornirea serverului. Un utilizator poate deține mai multe chat-uri simultan, fiecare cu propriul context și propriile tool-uri. La ștergerea cheii API, toate chat-urile asociate sunt eliminate automat prin `delete_all_chats()`.

Pipeline Plan-and-Execute cu Groq

Agentul de planificare implementează un pipeline în trei faze secvențiale, orchestrat direct în cod Python asincron fără un framework de graf de stare. Modelul de limbaj utilizat este configurat prin variabila `GROQ_MODEL` din fișierul `.env` și accesat printr-un client `AsyncGroq` instanțiat o singură dată la primul apel (*lazy singleton*).

Faza de planificare primește mesajul utilizatorului și lista scurtă a tool-urilor disponibile (nume și descriere) și solicită modelului Groq generarea unui plan JSON cu o listă ordonată de pași. Fiecare pas are un identificator numeric, o descriere și un tip: `"tool"` pentru pașii ce necesită execuția unui tool extern prin callback HTTP, respectiv `"llm"` pentru pașii de raționament pur, în care modelul generează text direct fără a apela niciun tool. Planificatorul primește și ultimele patru mesaje din istoricul conversației pentru a distinge cererile de continuare a dialogului de cererile ce necesită acțiuni noi. Promptul de sistem al planificatorului conține reguli explicite de *follow-up recognition*: dacă un tool a fost deja apelat cu succes și utilizatorul pune o întrebare despre rezultatele deja disponibile în istoric, planificatorul generează un singur pas `"llm"` în loc să reapeleze tool-ul.

Faza de execuție iterează secvențial prin pașii planului. Pentru pașii de tip `"tool"`, serverul localizează definiția tool-ului după nume, construiește un apel `POST` asincron către `callback_url` cu parametrii specificați în câmpul `input` al pasului și așteaptă rezultatul cu un timeout de 120 de secunde. Erorile de rețea sau HTTP sunt capturate și convertite în mesaje de eroare structurate, evitând întreruperea prematură a pipeline-ului. Pentru pașii de tip `"llm"`, modelul primește un prompt de raționament, sarcina originală și rezultatele pașilor anteriori și generează text în regim de streaming. Dacă modelul încearcă să apeleze un tool în timpul unui pas `"llm"` (comportament nedorit), sistemul aplică un mecanism de fallback în trei trepte: reîncercarea cu streaming dezactivat, reîncercarea cu un mesaj explicit care interzice apelurile de tool, și în final returnarea unui mesaj de eroare.

Faza de sinteză colectează rezultatele tuturor pașilor executați și solicită modelului Groq formularea unui răspuns final coerent, în regim de streaming. Răspunsul sintetizat este adăugat în istoricul chat-ului ca mesaj cu rolul `assistant` și transmis clientului ca eveniment final de tip `"final"`.

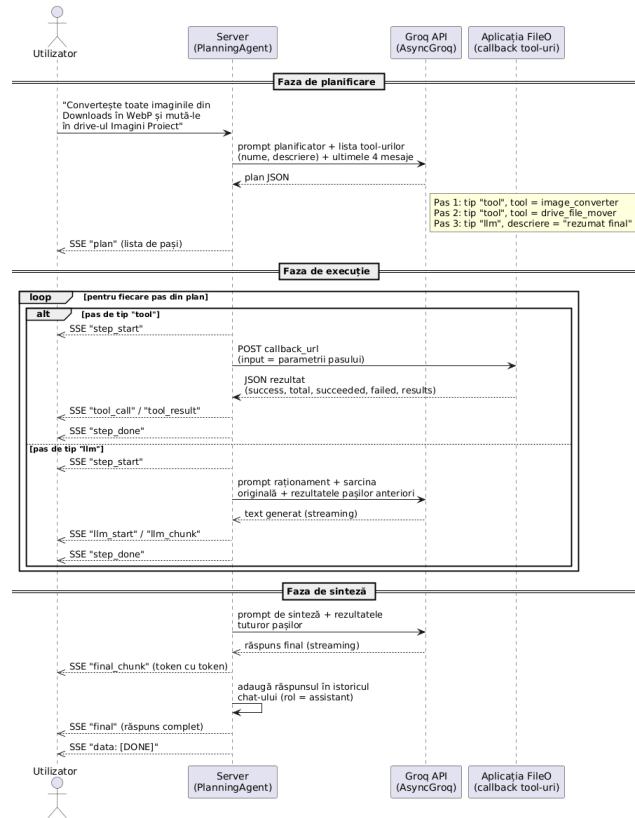


Figura 6.11: Diagrama de secvență: Trecerea pipeline a fazelor agentului

Streaming SSE și protocolul de evenimente

Toate răspunsurile agentului de planificare sunt livrate ca flux de *Server-Sent Events* (SSE), permițând interfeței utilizator să afișeze progresul execuției în timp real. Fiecare eveniment este serializat ca JSON pe o linie prefixată cu **data:** și urmat de două caractere newline, conform specificației SSE. Fluxul se încheie invariabil cu evenimentul special **data: [DONE]**.

Protocolul definește douăsprezece tipuri de evenimente cu semantică precisă: **status** pentru mesaje de stare generale, **plan** cu lista completă de pași generați, **step_start** și **step_done** la demararea și finalizarea fiecărui pas, **tool_call** cu parametrii de intrare ai apelului, **tool_result** sau **tool_error** cu rezultatul callback-ului, **llm_start**, **llm_chunk** și **final_chunk** pentru token-ii generați în streaming, **final** cu răspunsul complet și **error** pentru erorile fatale. Fiecare eveniment conține câmpul **message** cu un text human-readable în limba română, utilizat direct de interfața grafică fără procesare suplimentară.

Deoarece **EventSource** din browser suportă exclusiv cereri **GET**, aplicația FileO utilizează **fetch()** cu **ReadableStream** pentru a trimite mesajele prin **POST** și a consuma fluxul SSE. Proxy-ul Flask din aplicație preia streamul de la server și îl retransmite frontend-ului React fără buffering, adăugând headerul **X-Accel-Buffering: no** pentru

a dezactiva buffering-ul nginx.

Mecanismul de auto-recuperare

Dat fiind că stocarea chat-urilor este exclusiv în memorie, repornirea serverului invalidează toate chat-urile existente. Proxy-ul Flask din aplicație detectează automat un răspuns HTTP 404 la trimiterea unui mesaj și, în acel caz, creează transparent un nou chat, actualizează pointerul local `_ACTIVE_CHATS` și retrimite mesajul original. Utilizatorul nu percepe această operație, dar istoricul conversației anterioare repornirii este pierdut. Tool-urile sunt retrimise la fiecare mesaj, astfel că noul chat este funcțional imediat după recuperare.

6.3.6 Interfața web de administrare

Interfața web este o aplicație React single-page (SPA), compilată prin Vite cu TypeScript și stilizată cu Tailwind CSS. Aceasta este servită direct de FastAPI dintr-un director static, fără a necesita un server web separat. Un handler catch-all returnează `index.html` pentru orice rută necunoscută, ceea ce este necesar pentru funcționarea corectă a React Router în modul `BrowserRouter`. Fișierele statice sunt servite cu headerul `Cache-Control: no-cache` pentru a garanta că utilizatorii primesc întotdeauna versiunea curentă după un deployment.

Interfața expune cinci secțiuni principale: pagina principală cu informații despre platformă, secțiunea *Downloads* pentru descărcarea aplicației FileO și a containerului Docker, *Containers* pentru gestionarea cheilor API și vizualizarea sesiunilor active, *Wiki* cu documentația de utilizare și *Support*. Starea de autentificare este gestionată global printr-un context React (`AuthProvider`), evitând propagarea manuală a token-ului JWT prin props.

6.4 Testare

Validarea serverului a fost realizată prin două suite de teste de integrare end-to-end, scrise ca notebook-uri Jupyter și executate pentru infrastructura reală, containerul Docker al serverului și aplicația Flask locală, fără mock-uri, pentru a surprinde comportamentul efectiv al sistemului în condiții apropiate de producție.

Prima suită acoperă scenariile principale ale agentului de planificare: o interogare conversațională simplă, fără tool-uri, un apel `hello` fără aprobare, colectarea unui input prin `ask_user` cu aprobare automată, o conversie de imagine prin `image_converter`, precum și o conversație multi-turn care testează mecanismul de *follow-up recognition*. Suplimentar, suita validează recunoașterea vizuală prin Groq Llama-4-Scout-17B, obținând un scor mediu de keywords de 94% (prag 70%), și transcrierea audio prin

Whisper Large V3 pe cinci fișiere din dataset-ul LJSpeech, cu un WER mediu de 0,070 (prag $\leq 0,30$), ambele rezultate confirmând acuratețea modelelor în scenarii reale de utilizare prin agent.

A doua suită validează modelele AI expuse prin gateway-ul serverului, independent de logica agentului. Whisper a fost testat separat, obținând un WER mediu de 0,082 pe CPU, cu un RTF (*real-time factor*) de 7,91. Swin2SR a fost testat la rezoluții de intrare de 64, 128 și 256 de pixeli, confirmând un upscaling $\times 2$ corect pentru fiecare dimensiune. Gateway-ul LLM Groq a fost validat în toate cele trei moduri de generare expuse în mod sincron, streaming SSE și batch, la diferite temperaturi (parametru care controlează gradul de aleatoriu al textului generat: valori mici produc răspunsuri mai deterministe și concentrate pe cele mai probabile cuvinte, valori mari produc răspunsuri mai variate și creative), verificând atât corectitudinea răspunsurilor cât și stabilitatea conexiunilor în timpul streaming-ului.

Capitolul 7

Manualul Utilizatorului

Acest capitol descrie pașii necesari pentru a accesa și utiliza platforma FileO, compusă din serverul de backend și aplicația desktop. Nu sunt necesare cunoștințe tehnice, fiecare operație este ilustrată cu capturi de ecran ale interfeței reale.

7.1 Accesul la Aplicație

7.1.1 Autentificarea pe Server

Înainte de a utiliza funcționalitățile AI sau agentul de planificare, utilizatorul trebuie să își creeze un cont pe server și să se autentifice prin interfața web de administrare.

1. Se deschide în browser adresa serverului (ex. `http://localhost:8000`).
2. Se face clic pe butonul **Register** și se completează adresa de email și o parolă de minimum 8 caractere, apoi se confirmă.
3. După înregistrare, utilizatorul va fi redirecționat automat la pagina principală, unde apare mesajul de bun venit și datele contului.
4. La accesările ulterioare se folosește butonul **Login** cu aceleași credențiale.

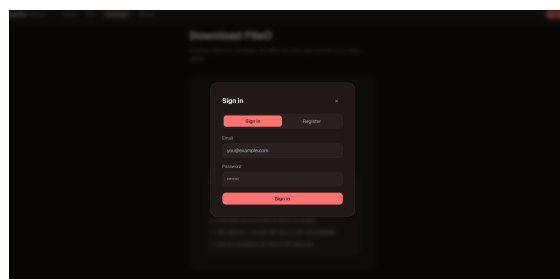


Figura 7.1: Pagina de autentificare a serverului

7.1.2 Instalarea Aplicației

Aplicația FileO este distribuită ca executabil Windows și nu necesită instalare suplimentară de dependențe.

1. Din interfața web, se navighează la secțiunea **Downloads** și se descarcă arhiva aplicației.
2. Conținutul se dezarchivează într-un folder la alegere (ex. `C:\FileO`).
3. Aplicația se pornește executând fișierul `FileO.exe`. La prima pornire, sistemul de operare poate afișa un avertisment SmartScreen; se alege *Run anyway*.
4. Aplicația pornește și deschide automat fereastra principală. Backend-ul Python este lansat în fundal în mod transparent.

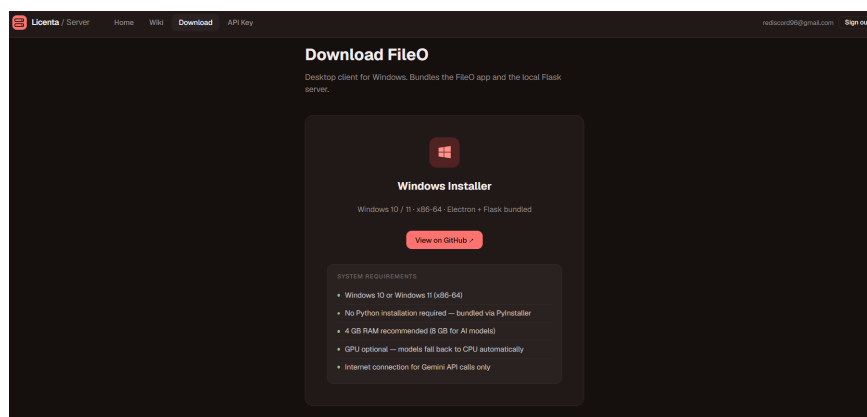


Figura 7.2: Descărcarea aplicației FileO din interfața web

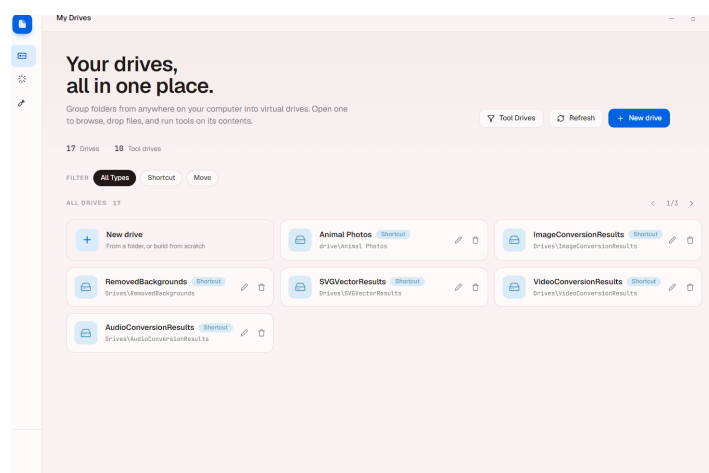


Figura 7.3: Fereastra principală a aplicației FileO la prima pornire

7.1.3 Acces la Interfața API

Cheia API conectează aplicația FileO la serverul de agent și este necesară pentru a putea folosi secțiunea Agent.

1. În interfața web, utilizatorul se autentifică și navighează la secțiunea **Account** sau **API Key**.
2. Se apasă **Generate API Key**; cheia va fi afișată o singură dată și trebuie copiată imediat.
3. În aplicația FileO se deschide **Settings** (pictograma din bara laterală stângă) și cheia se introduce în câmpul *API Key*.
4. Se completează câmpul *Server URL* cu adresa serverului (ex. `http://localhost:8000`) și se apasă **Save**.
5. Indicatorul de stare din secțiunea Agent va deveni verde, confirmând conexiunea.

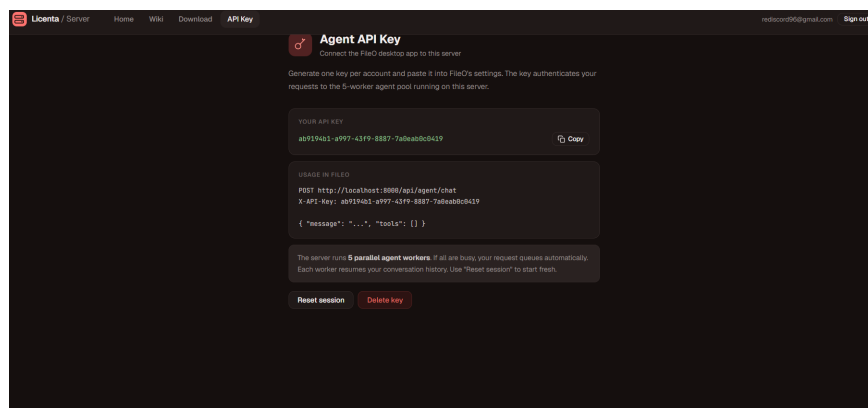


Figura 7.4: Generarea cheii API din interfața web a serverului

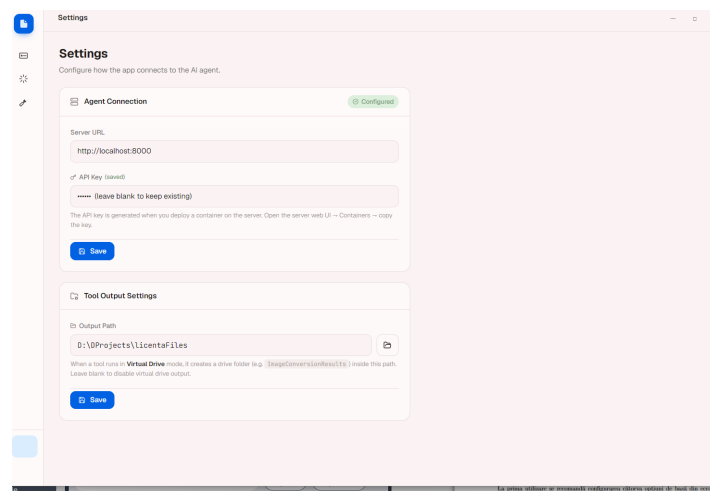


Figura 7.5: Configurarea cheii API în aplicația FileO

7.2 Secțiunile din Aplicație

7.2.1 Setări Inițiale

La prima utilizare se recomandă configurarea câtorva opțiuni de bază din ecranul **Settings** (pictograma roată din bara laterală).

- **Output Path**, folderul implicit unde instrumentele vor salva fișierele procesate. Se apasă *Browse* și se selectează un folder de pe disc (ex. `D:\File00output`).
- **Server URL și API Key**, completate conform pașilor din secțiunea 7.1.3.
- **Temă vizuală**, butonul lună/soare din bara laterală comută între tema deschisă și cea închisă.

7.2.2 Drive Tab

Secțiunea **My Drives** permite organizarea fișierelor de pe disc în drive-uri virtuale grupate pe categorii.

1. Se apasă **New Drive** și se introduce un nume descriptiv (ex. *Imagini Proiect*).
2. Se selectează folderul sursă pe care aplicația îl va scana și tipurile de fișiere de inclus (ex. `.jpg`, `.png`).
3. Se alege acțiunea: *Shortcuts* (creează scurtături fără a muta originalele) sau *Move* (mută fișierele în drive).
4. Se apasă **Create**; drive-ul apare în lista din stânga cu numărul de fișiere identificate.
5. Se face clic pe un drive pentru a vedea conținutul și a naviga prin fișierele grupate.

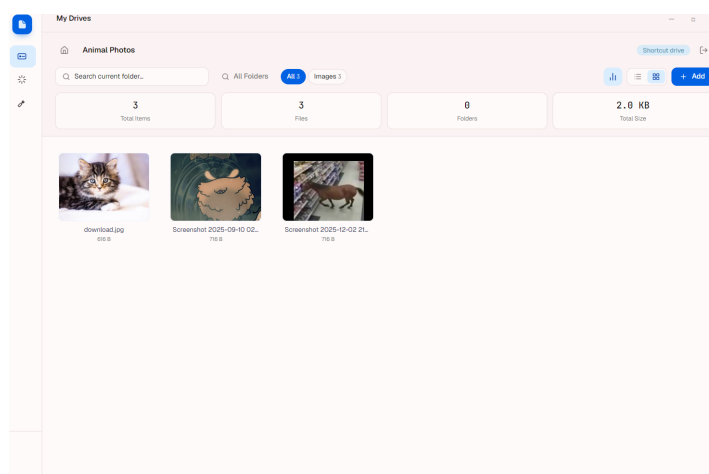


Figura 7.6: Secțiunea My Drives, vizualizare drive virtual

7.2.3 Agent Tab

Secțiunea **Agent** oferă o interfață de chat prin care utilizatorul poate descrie în limbaj natural sarcini complexe pe care agentul le va planifica și executa automat.

1. Se verifică că serverul rulează și cheia API este configurată (indicatorul de stare este verde).
2. Cererea se scrie în câmpul de mesaj și se apasă *Enter* sau butonul de trimitere. Exemplu: „*Convertește toate imaginile din folderul Downloads în format WebP și mută-le în drive-ul Imagini Proiect*”.
3. Agentul afișează în timp real gândirea sa internă (colapsabilă) și fiecare tool pe care intenționează să îl apeleze.
4. Când un tool necesită aprobare, apare un dialog cu parametrii propuși. Parametrii pot fi modificați dacă este necesar, după care se apasă **Approve** sau **Reject**.
5. La finalizare, agentul afișează un sumar al acțiunilor efectuate.

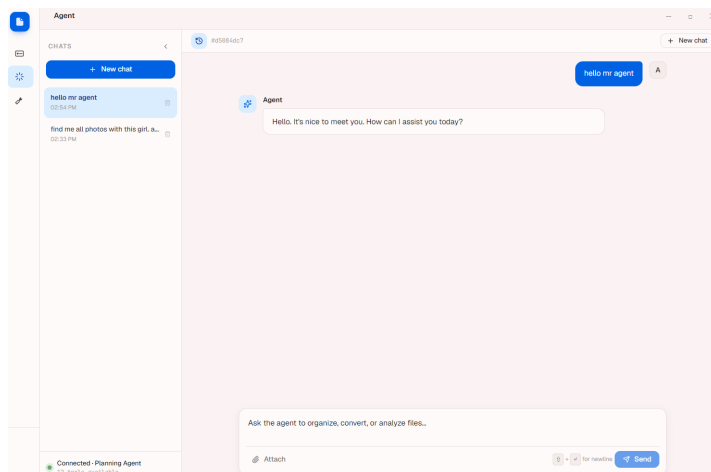


Figura 7.7: Secțiunea Agent, conversație activă cu agentul de planificare

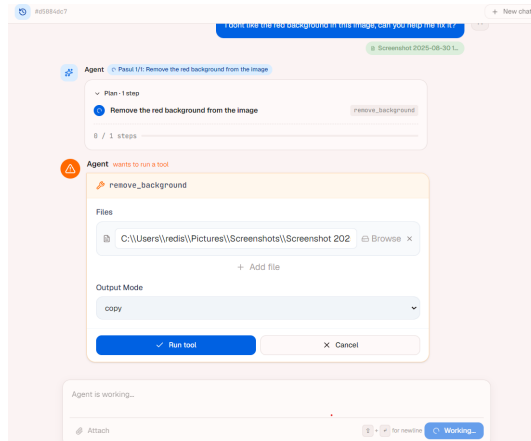


Figura 7.8: Dialogul de aprobare a execuției unui instrument

7.2.4 Tool Tab

Secțiunea **Tools** permite rularea manuală a oricărui instrument disponibil, fără a implica agentul.

1. Se navighează la **Tools** din bara laterală. Instrumentele sunt afișate ca un grid organizat pe categorii (Image, Audio, Video, Documents etc.). Se folosesc filtrele pill din partea de sus pentru a restrânge lista.
2. Se face click pe un instrument pentru a vedea descrierea, formatul de fișiere acceptat și pașii de utilizare recomandați.
3. Se apasă **Run Tool** pentru a deschide formularul de execuție.
4. Se completează câmpurile necesare: se selectează fișierele de intrare (*Browse* sau *From Drive*), se alege formatul de ieșire și modul de salvare (*Copy*, *Replace* sau *Virtual Drive*).
5. Se apasă **Run** și se așteaptă finalizarea. Instrumentele AI (Image Enhancer, Audio Transcriber) afișează o bară de progres în timp real. La final, lista de rezultate indică fișierele procesate cu succes și eventualele erori.

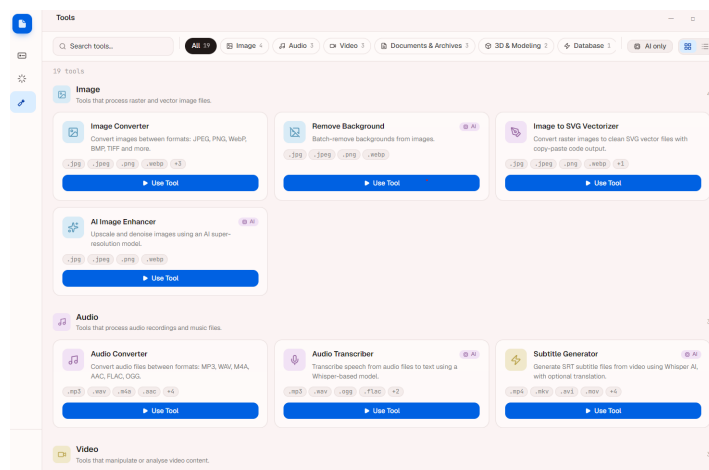


Figura 7.9: Secțiunea Tools, grila de instrumente organizată pe categorii

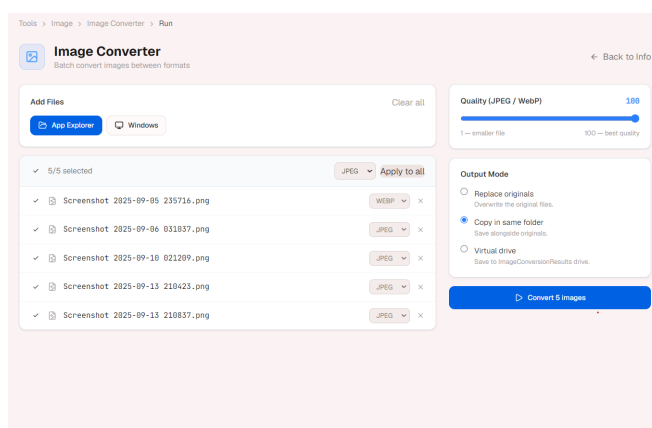


Figura 7.10: Formularul de execuție al unui instrument

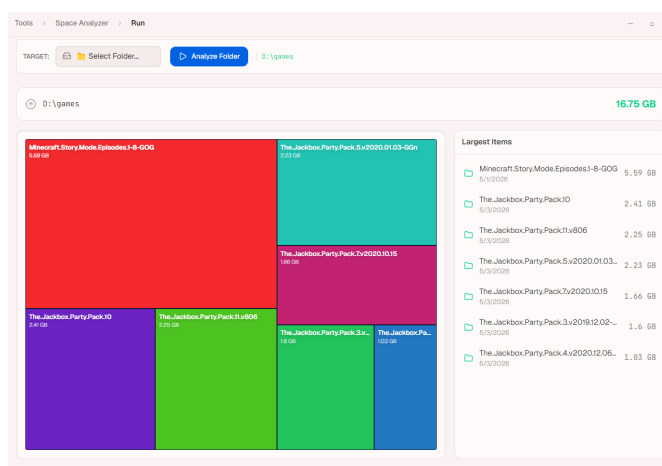


Figura 7.11: Rezultatele după execuția unui instrument

Capitolul 8

Concluzii

Prezenta lucrare propune o abordare modernă și integrată a uneia dintre problemele frecvent întâlnite în rândul utilizatorilor obișnuiți: organizarea, prelucrarea și transformarea fișierelor locale într-un mod eficient, sigur și accesibil. Soluția dezvoltată reunește tehnologii de sistem de nivel apropiat de nivelul de stocare brut, cu capabilități avansate de inteligență artificială, materializându-se sub forma platformei FileO, o aplicație desktop destinată gestionării inteligente a fișierelor.

Fundamentul tehnic al lucrării se sprijină pe doi piloni principali. Pe de o parte, indexarea rapidă a sistemului de fișiere prin interogarea directă a structurii NTFS Master File Table (Capitolul 2) permite scanarea unor volume mari de date în fracțiuni de secundă, fără a recurge la parcurgerea recursivă clasică a directoarelor. Pe de altă parte, integrarea modelelor de inteligență artificială (Capitolul 3), concret, modelul Swin2SR pentru suprarezoluție și restaurare de imagini, respectiv Whisper Large V3 pentru transcriere audio și generare de subtitrări, conferă aplicației capabilități de procesare care depășesc limitele instrumentelor tradiționale.

Arhitectura sistemului este structurată pe două componente distincte și complementare. Serverul, expus ca serviciu web prin FastAPI și rulat local, furnizează atât modelele de inteligență artificială prin intermediul unui API REST, cât și agentul de planificare bazat pe paradigma Plan-and-Execute, orchestrat direct în cod Python asincron, fără un framework de graf de stare. Acesta din urmă permite utilizatorilor să descrie în limbaj natural sarcini complexe, multi-pas, pe care agentul le planifică și le execută automat, solicitând aprobarea umană înainte de fiecare acțiune cu efect ireversibil. Aplicația desktop FileO, construită pe stiva Electron + React + TypeScript cu un micro-backend Flask integrat, pune la dispoziția utilizatorului un catalog de instrumente organizat în mai multe categorii: Image, Audio, Video, Documente 3D, Computer Tools, și o interfață de tip chat pentru interacțiunea cu agentul. Utilizatorii pot converti, comprima, vectoriza, transcrie sau analiza fișiere fără a fi nevoiți să încarce date în servicii web externe, beneficiind astfel de un nivel sporit de confidențialitate și control asupra propriilor date.

În ciuda funcționalităților demonstrate, platforma FileO se află în prezent la stadiul de *proof of concept* și reprezintă, în primul rând, o dovadă a potențialului pe care îl poate atinge o aplicație de gestionare a fișierelor susținută de inteligență artificială. Există mai multe direcții naturale de evoluție care ar putea transforma proiectul într-un produs viabil pe piață.

O primă direcție vizează consolidarea infrastructurii serverului. Configurația actuală este dimensionată pentru un număr restrâns de utilizatori concurenți; o iterație viitoare ar presupune migrarea pe o instanță de calcul cu resurse GPU dedicate, ceea ce ar permite rularea simultană a mai multor modele de inteligență artificială, reducerea timpilor de inferență și servirea unui număr mai mare de clienți prin intermediul aceluiași API.

O a doua direcție, cu impact semnificativ asupra adoptării pe scară largă, este implementarea unui sistem de extensii inspirat din ecosistemul Visual Studio Code. Prin expunerea unui API public de plugin-uri, terți dezvoltatori ar putea contribui cu instrumente proprii, direct integrabile în catalogul aplicației, fără a necesita modificarea codului de bază. Un astfel de mecanism ar stimula formarea unei comunități active de contribuiitori, ar extinde spectrul de cazuri de utilizare acoperite și ar permite fiecărui utilizator să personalizeze experiența în funcție de nevoile specifice ale fluxului său de lucru.

Nu în ultimul rând, interfața utilizator, atât a aplicației desktop, cât și a panoului de administrare web al serverului, poate fi rafinată pentru a reduce bariera de adoptare în rândul utilizatorilor fără experiență tehnică. Simplificarea fluxurilor de configurare inițială, adăugarea unor ghiduri contextuale și îmbunătățirea prezentării rezultatelor procesării sunt măsuri care ar crește semnificativ gradul de utilizabilitate al platformei.

Ansamblul acestor îmbunătățiri ar putea constitui punctul de pornire al unei noi iterații a proiectului, cu un potențial real de poziționare pe piața aplicațiilor de productivitate augmentate de agenți inteligenți.

Bibliografie

- [AADC25] Mohamad Abou Ali, Fadi Dornaika, and Jinan Charafeddine. Agentic ai: a comprehensive survey of architectures, applications, and future directions. *Artificial Intelligence Review*, 59(1):11, 2025.
- [Adm26] M365 Admin. Microsoft copilot (microsoft 365): Use copilot with onedrive files in file explorer and activity center, 2026.
- [AKD25] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access*, 13:18912–18936, 2025.
- [Ant26] Anthropic. Claude cowork: Anthropic’s agentic ai for knowledge work, 2026.
- [AVW09] Mamoun Alazab, Sitalakshmi Venkatraman, and Paul Watters. Effective digital forensic analysis of the ntfs disk image. *Ubiquitous Computing and Communication Journal*, 4(1):551–558, 2009.
- [BBA25] Andrew A Borkowski and Alon Ben-Ari. Multiagent ai systems in health care: envisioning next-generation intelligence. *Federal Practitioner*, 42(5):188, 2025.
- [BF81] Bruce G Buchanan and Edward A Feigenbaum. Dendral and meta-dendral: Their applications dimension. In *Readings in artificial intelligence*, pages 313–322. Elsevier, 1981.
- [BG03] Marina Bosi and Richard E. Goldberg. *Introduction to Digital Audio Coding and Standards*. Kluwer Academic Publishers, 2003.
- [BJH⁺22] Muhammad Babar, Mian Ahmad Jan, Xiangjian He, Muhammad Usman Tariq, Spyridon Mastorakis, and Ryan Alturki. An optimized iot-enabled big data analytics architecture for edge–cloud computing. *IEEE Internet of Things Journal*, 10(5):3995–4005, 2022.
- [BK19] Senay Baydas and Bulent Karakas. Defining a curve as a bezier curve. *Journal of Taibah University for Science*, 13(1):522–528, 2019.

- [Bra99] Karlheinz Brandenburg. Mp3 and aac explained. In *Proceedings of the AES 17th International Conference: High-Quality Audio Coding*. Audio Engineering Society, 1999.
- [Car05] Brian Carrier. *File system forensic analysis*. Addison-Wesley Professional, 2005.
- [CCBT22] Marcos V. Conde, Ui-Jin Choi, Maxime Burchi, and Radu Timofte. Swin2sr: Swinv2-based intermediate-layer restorer for low-level vision, 2022.
- [CL25] Yuezhang Chen and Jiacheng Liu. Ai-driven marketing strategy optimization in the digital economy: A machine learning approach. In *Proceedings of the 2025 International Conference on Digital Economy and Intelligent Computing*, pages 239–242, 2025.
- [CMH⁺18] Yue Chen, Debargha Murherjee, Jingning Han, Adrian Grange, Yaowu Xu, Zoe Liu, Sarah Parker, Cheng Chen, Hui Su, Urvang Joshi, et al. An overview of core coding tools in the av video codec. In *Picture Coding Symposium (PCS)*, pages 41–45. IEEE, 2018.
- [Com26] Microsoft Tech Community. Computer-using agents in microsoft copilot studio are now generally available, 2026.
- [CSC⁺21] Xiaoyi Chen, Ahmed Salem, Dingfan Chen, Michael Backes, Shiqing Ma, Qingni Shen, Zhonghai Wu, and Yang Zhang. Badnl: Backdoor attacks against nlp models with semantic-preserving improvements. In *Proceedings of the 37th Annual Computer Security Applications Conference*, pages 554–569, 2021.
- [CSM⁺23] Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krasheninnikov, Lauro Langosco, Zhonghao He, Yawen Duan, Micah Carroll, et al. Harms from increasingly agentic algorithmic systems. In *Proceedings of the 2023 ACM conference on fairness, accountability, and transparency*, pages 651–666, 2023.
- [dCLCT⁺24] Erik de Castro Lopo, Josh Coalson, Martijn Thoby, et al. Rfc 9639: Free lossless audio codec (flac). Internet Engineering Task Force (IETF), 2024.
- [DGGR23] Adrien Dubettier, Tanguy Gernot, Emmanuel Giguet, and Christophe Rosenberger. File type identification tools for digital investigations. *Forensic Science International: Digital Investigation*, 46:301574, 2023.

- [FB96] Ned Freed and Nathaniel Borenstein. Multipurpose internet mail extensions (mime) part one: Format of internet message bodies. Technical report, 1996.
- [FSpz⁺24] Mathieu Fenniak, Matthew Stamy, pubpub zz, Martin Thoma, Matthew Peveler, and exiledkingcc. The pypdf library, 2024. See <https://pypdf.readthedocs.io>.
- [FYJ⁺25] Tianyu Fan, Yuhao Yang, Yangqin Jiang, Yifei Zhang, Yuxuan Chen, and Chao Huang. Ai-trader: Benchmarking autonomous agents in real-time financial markets. *arXiv preprint arXiv:2512.10971*, 2025.
- [Gar26] Jean-Philippe Garnier. Agentic hives: Equilibrium, indeterminacy, and endogenous cycles in self-organizing multi-agent systems. *arXiv preprint arXiv:2603.00130*, 2026.
- [Gat23] Daniel Gatis. rembg: A python tool to remove images background, 2023. Versiunea 2.0.28, bazată pe arhitectura U2-Net.
- [GDGG17] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. arxiv 2017. *arXiv preprint arXiv:1708.06733*, 2017.
- [gTN26] gHacks Tech News. Windows 11 leak suggests copilot is moving deeper into file explorer, 2026.
- [IN24] Yoichi Ishibashi and Yoshimasa Nishimura. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183*, 2024.
- [KR25] Nareta Kataria and Khalid Rasheed. Ai-driven recruitment and hiring automation through cv scanning and ai agents. *Sukkur IBA Journal of Computing and Mathematical Sciences*, 9(02):54–61, 2025.
- [LB02] William S. Lovegrove and David F. Brailsford. Document analysis of pdf files: methods, results and implications. In *Proceedings of the 2002 ACM Symposium on Document Engineering, DocEng '02*, pages 105–114, New York, NY, USA, 2002. ACM.
- [Mar26] MarkTechPost. Anthropic releases cowork as claude’s local file system agent for everyday work, 2026.
- [MB08] Kenton McHenry and Peter Bajcsy. An overview of 3d data content, file formats and viewers. *National Center for Supercomputing Applications*, 1205(22):1–21, 2008.

- [MBG⁺13] Deb Mukherjee, Jim Bankoski, Adrian Grange, Jingning Han, John Koleszar, Paul Wilkins, Yaowu Xu, and Ronald Bultje. Vp9 bitstream decoding process specification. Google Technical Report, 2013.
- [McA24] Derek McAuley. Resilient machine learning frameworks: Strategies for mitigating data poisoning vulnerabilities. *International Journal of Digital Innovation*, 5(1), 2024.
- [Nie07] Ernst Niebur. Saliency map. *Scholarpedia*, 2(8):2675, 2007.
- [One] OneRep. Is cloudconvert safe? what you need to know before uploading files.
- [PMEM⁺23] Sebastian Porsdam Mann, Brian D Earp, Nikolaj Møller, Suren Vynn, and Julian Savulescu. Autogen: A personalized large language model for academic enhancement—ethics and proof of principle. *The American Journal of Bioethics*, 23(10):28–41, 2023.
- [Pur26] PureVPN. What is wiztree and how does it work? a beginner’s guide, 2026.
- [QHL⁺24] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4):1–40, 2024.
- [QZH⁺20] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Osmar R Zaiane, and Martin Jagersand. U2-net: Going deeper with nested u-structure for salient object detection. *Pattern recognition*, 106:107404, 2020.
- [Ram26] Toni Ramchandani. Claude cowork: The complete guide to anthropic’s ai desktop agent. Medium – Data And Beyond, 2026.
- [RKX⁺22] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2022.
- [SEA26] Christopher Stovah, Kedi Wagobera Edgar, and Adekunle Adegboye. Agentic ai for real-time fraud prevention in digital finance. *International Journals For American Academic Publisher*, pages 24–32, 2026.
- [Sel03] Peter Selinger. Potrace: a polygon-based tracing algorithm, 2003.

- [Sho12] Edward Shortliffe. *Computer-based medical consultations: MYCIN*, volume 2. Elsevier, 2012.
- [SLY⁺22] Yundi Shi, Piji Li, Changchun Yin, Zhaoyang Han, Lu Zhou, and Zhe Liu. Promptattack: Prompt-based attack for language models via gradient search. In *CCF International Conference on Natural Language Processing and Chinese Computing*, pages 682–693. Springer, 2022.
- [Sma] Smallpdf. Pdf converter – convert pdfs online to and from any format.
- [SOHW12] Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (hevc) standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 22(12):1649–1668, 2012.
- [SYY⁺24] Shibo Sheng, Hui Yin, Ying Yang, Aixin Chong, and Hua Huang. Dunet: Dense u-blocks network for fine-grained crack detection. *Signal, Image and Video Processing*, 18(2):1929–1938, 2024.
- [TD25] Tom Taulli and Gaurav Deshmukh. Crewai. In *Building Generative AI Agents: Using LangGraph, AutoGen, and CrewAI*, pages 103–145. Springer, 2025.
- [Tec26] TechRepublic. Leaked windows 11 feature shows copilot moving into file explorer, 2026.
- [TMPB23] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.
- [TZZ⁺24] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 599–621, 2024.
- [Ven26] VentureBeat. Anthropic launches cowork, a claude desktop agent that works in your files – no coding required, 2026.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach,

- R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [VVT12] Jean-Marc Valin, Koen Vos, and Timothy Terriberry. Rfc 6716: Definition of the opus audio codec. Internet Engineering Task Force (IETF), 2012.
- [Wiza] WizTree. Wiztree – about.
- [Wizb] WizTree. Wiztree – the fastest disk space analyzer.
- [WSBL03] Thomas Wiegand, Gary J. Sullivan, Gisle Bjontegaard, and Ajay Luthra. Overview of the h.264/avc video coding standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7):560–576, 2003.
- [XCG⁺25] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [YRS⁺24] Gokul Yenduri, Muthukumaran Ramalingam, G Chemmalar Selvi, Y Supriya, Gautam Srivastava, Praveen Kumar Reddy Maddikunta, G Deepti Raj, Rutvij H Jhaveri, B Prabadevi, Weizheng Wang, et al. Gpt (generative pre-trained transformer)—a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. *IEEE access*, 12:54608–54649, 2024.